

Statistica descrittiva

Dispensa del corso *Analisi dei dati con applicazioni informatiche* – Moduli 1, 2 e 3

Ottavia M. Epifania

Invalid Date

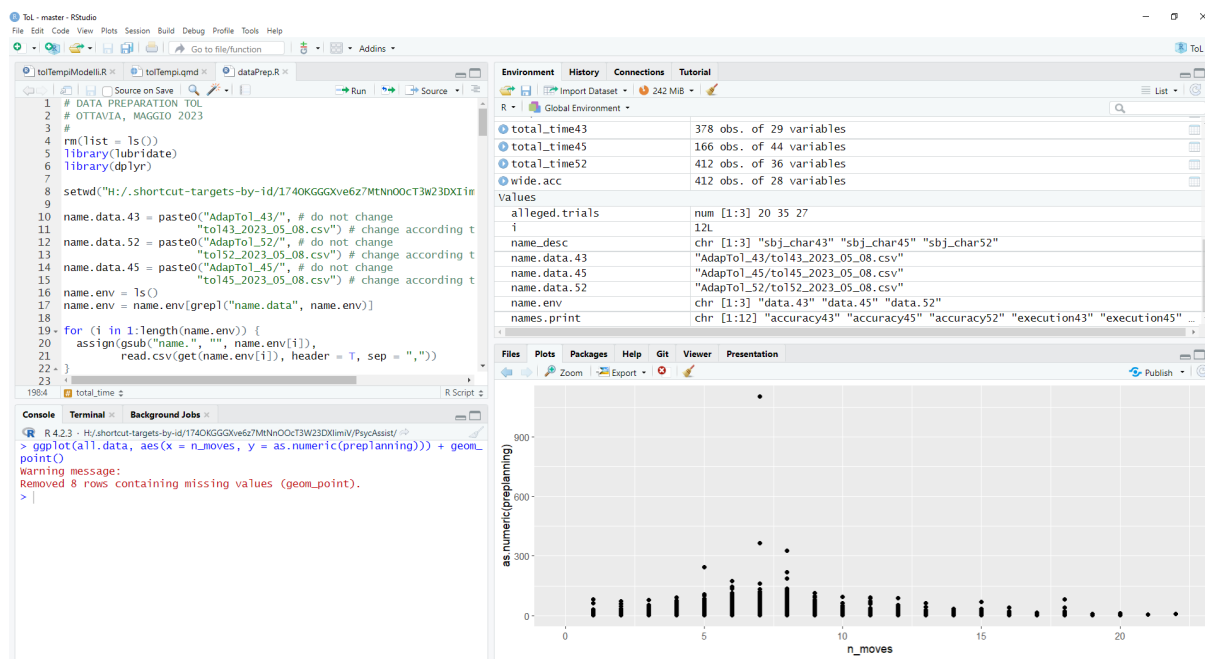
Indice

1	Modulo 1 – Introduzione a R	3
1.1	RStudio	3
1.2	Working directories (wd)	4
1.3	Script e reporting	6
1.4	Reporting con quarto	6
1.5	Elementi base	9
1.6	Pacchetti e Funzioni	12
1.7	Operatori	14
1.8	Tavole di verità	16
1.9	R ed errori	17
1.10	Vettori	18
1.11	Matrici	22
1.12	Dataframe	24
1.13	Liste	26
1.14	Importare dati esterni	27
1.15	Sintesi delle strutture tabellari	29
2	Modulo 2 – Scale di misura e statistica descrittiva univariata	32
2.1	Variabili	32
2.2	Frequenze	37
2.3	Indici di posizione	39
2.4	Perché i grafici	43
2.5	Grafici	43
2.6	Esempi	48
3	Modulo 3 – Statistica descrittiva bivariata	52
3.1	Variabile statistica doppia	52
3.2	Tabella a doppia entrata – Tavola di contingenza	52
3.3	Connessione (Variabili qualitative)	56
3.4	Indipendenza stocastica e massima dipendenza	59
3.5	Associazione (variabili quantitative)	61
3.6	Previsione variabili quantitative	69
3.7	Materiale supplementare	82
4	Funzioni riassuntive dei moduli	86
4.1	Operatori	86
4.2	Valori di verità	86
4.3	Operatori e funzioni utili	87
4.4	Convertitori	87
4.5	Strumenti	87
4.6	Funzioni matematiche	88
4.7	Matrici	88
4.8	Liste e Data Frame	89
4.9	Distribuzioni di frequenza	89

4.10	Funzioni grafiche	89
4.11	Associazione variabili categoriali	90
4.12	Regressione lineare	90

1 Modulo 1 – Introduzione a R

1.1 RStudio



Ambiente/Environment (Pannello in alto a dx). Contiene tutti gli oggetti che sono stati creati fino a quel momento nell'ambiente di lavoro. Viene automaticamente suddiviso in oggetti di tipo vettoriale o scalare (**values**) e oggetti più complessi come dataframe e liste.

Script. (Pannelli in alto a sinistra). File di tipo testuale con estensione `.R` dove salvare il codice usato per generare gli oggetti presenti nell'ambiente. Si possono combinare codice e commenti (righe che non vengono eseguite e interpretate come codice, sono le righe precedute dal simbolo `#`, vedi Listing 1.1). Per eseguire i comandi negli script bisogna usare la combinazione di tasti `ctrl + invio` (per il mac, `cmd + invio`)

Console. (Pannello in basso a sinistra). Pannello di lavoro dove i lavori vengono immediatamente eseguiti immediatamente quando si clicca invio. Le righe di comando non vengono salvate ma sono sempre accessibili premendo con la freccina in su della tastiera. Nella console appaiono anche i risultati dei codici eseguiti nello script.

Plot/Files. (Pannello in basso a dx). Pannello composto da diversi pannelli. Nel pannello **Plots** compaiono i grafici risultanti dalle analisi statistiche svolte negli script. Nel pannello **Files** appaiono tutti i file e le cartelle disponibili in una data working directory (Vedi Section 1.2 e Section 1.2.2).

Listing 1.1 Codice con commento

```
# codice per assegnare il numero 25 alla variabile x
x <- 25
```

Console

I comandi nella console vengono eseguiti con click su Invio
Il codice non viene salvato e scompare sequenzialmente.
L'output appare immediatamente nella console.
Per fare lo switch alla console → `ctrl + 2`

Script

Si possono salvare gli script di codice
Per far “correre” qualsiasi comando nello script → `Ctrl + Invio` (o `cmd + Enter`)
L'output appare nella console.
Per fare lo switch allo script → `ctr + 1`

1.2 Working directories (wd)

Working directory. La cartella dentro cui sta lavorando R in quel momento. Diventa di fondamentale importanza perché è dove R si aspetta che ci siano i file (e.g., i dati) con cui volete lavorare

1.2.1 Hard way

Listing 1.2 Funzione per ottenere la wd, ovvero la cartella dentro cui sta lavorando R

```
# per sapere dove sta lavorando R
getwd()
```

```
[1] "C:/Users/Ottavia/Documents/GitHub/stpc-analisi-dati"
```

La wd si può cambiare con il comando `setwd()`:

Listing 1.3 Funzione per ottenere cambiare la wd dentro cui sta lavorando R. Dentro la funzione, va messo il percorso completo entro cui si vuole spostare la cartella di lavoro tra virgolette, come nell'esempio sottostante.

```
setwd("C:/Users/Ottavia/Documents/altro-percorso")
```

1.2.2 Easy Way: R Project

Crea automaticamente una working directory (ovvero una cartella) in cui potete trovare tutto il materiale necessario per delle analisi. Questo vi evita di dover pensare alla gestione delle working directories.

Struttura tipica

```
my-project/  
|  
+-- index.qmd  
+-- laboratorioTemplate.qmd  
+-- myScript.R  
+-- my-project.Rproj  
+-- data/  
|   +-- mydata.csv
```

Nell'esempio precedente, la cartella di ordine superiore è la cartella **my-project**, ovvero la cartella che contiene tutto il vostro progetto. Al suo interno si trovano i seguenti file, a diversi livelli:

1. Primo Livello: `index.qmd`, `laboratorioTemplate.qmd`, `myScript.R` (file di quarto con estensione `qmd`, file R con estensione `R`) e una cartella `data`. Il file `my-project.Rproj` è il file del vostro progetto R con estensione `Rproj`.
2. Secondo Livello: all'interno della cartella `data` si trova il file `mydata.csv`. Per accedere a questo file di dati da un file contenuto nella cartella di ordine superiore, bisogna semplicemente comunicare a R che si trova nella cartella `data`, "`data/mydata.csv`".

1.2.2.1 Creare un nuovo progetto

Da **File** → **New Project** e seguire i seguenti step:

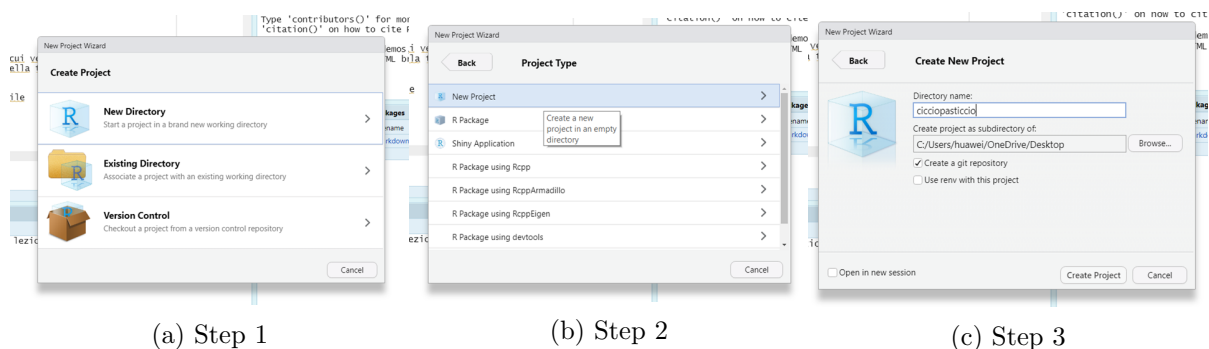


Figure 1.1: Creare un progetto R in 3 step

In Figure 1.1a selezionare l'opzione **New directory**.

In Figure 1.1b selezionare l'opzione **New Project** (ignorare tutte le altre opzioni).

In Figure 1.1c dare un nome al progetto (e.g., `corso-analisi`) e scegliete la cartella entro cui salvarlo (consigliato: I Documenti del vostro computer).

Figure 1.2 illustra il controllo per capire se effettivamente siete dentro al progetto R. Se non siete dentro il progetto R, andate a cercarlo nella cartella dove lo avete creato e fate doppio click sul file con estensione `Rproj`.

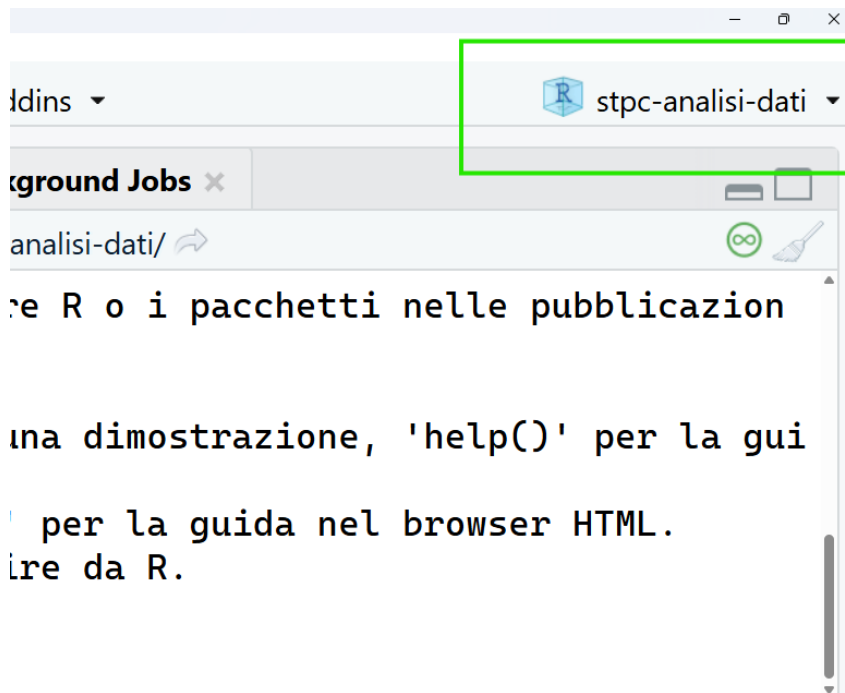


Figure 1.2: Indica il nome del progetto

1.3 Script e reporting

Figure 1.3 illustra come creare un nuovo script, dove è possibile eseguire e salvare una serie di comandi.

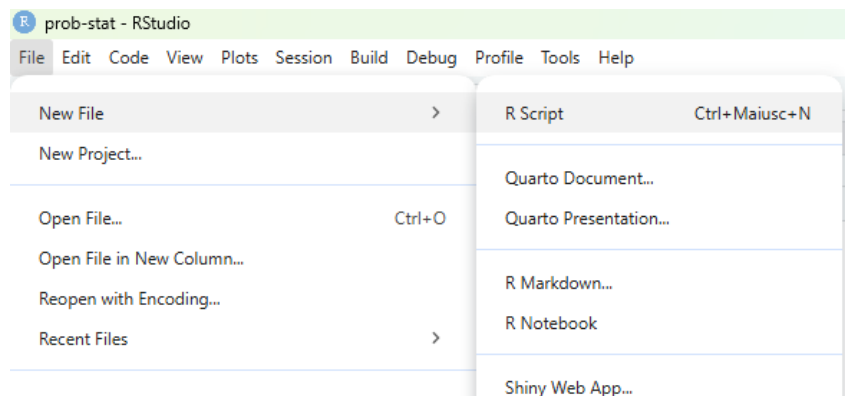


Figure 1.3: Creare un nuovo script

In alternativa, si può usare la sequenza di tasti: **Ctrl + shift + n** (cmd + shift + n)

Per salvare uno script, bastare cliccare sull'icona del salvataggio.

1.4 Reporting con quarto

quarto è un markup language integrato con RStudio.

Permette la creazione di documenti (sia pdf sia html) che possono contenere chunk di codice, ovvero delle parti dove viene eseguito il codice R, per ottenere risultati, tabelle e grafici.

In questo corso, useremo quarto per tutti gli esercizi. Vi verranno forniti dei template, specie all'inizio, nella speranza che riusciate a modificarli e a diventare via via più autonome nel suo utilizzo.

Una parte dell'esame prevede l'utilizzo di quarto per lo svolgimento degli esercizi. le esercitazioni in itinere sono da svolgere su quarto.

A [questo link](#) trovate maggiori informazioni su quarto e una guida veloce sul suo funzionamento.

Figure 1.4a crea un nuovo file quarto, mentre Figure 1.4b illustra come specificare varie opzioni del documento (titolo, autrice).

Figure 1.4: Crea un nuovo file quarto

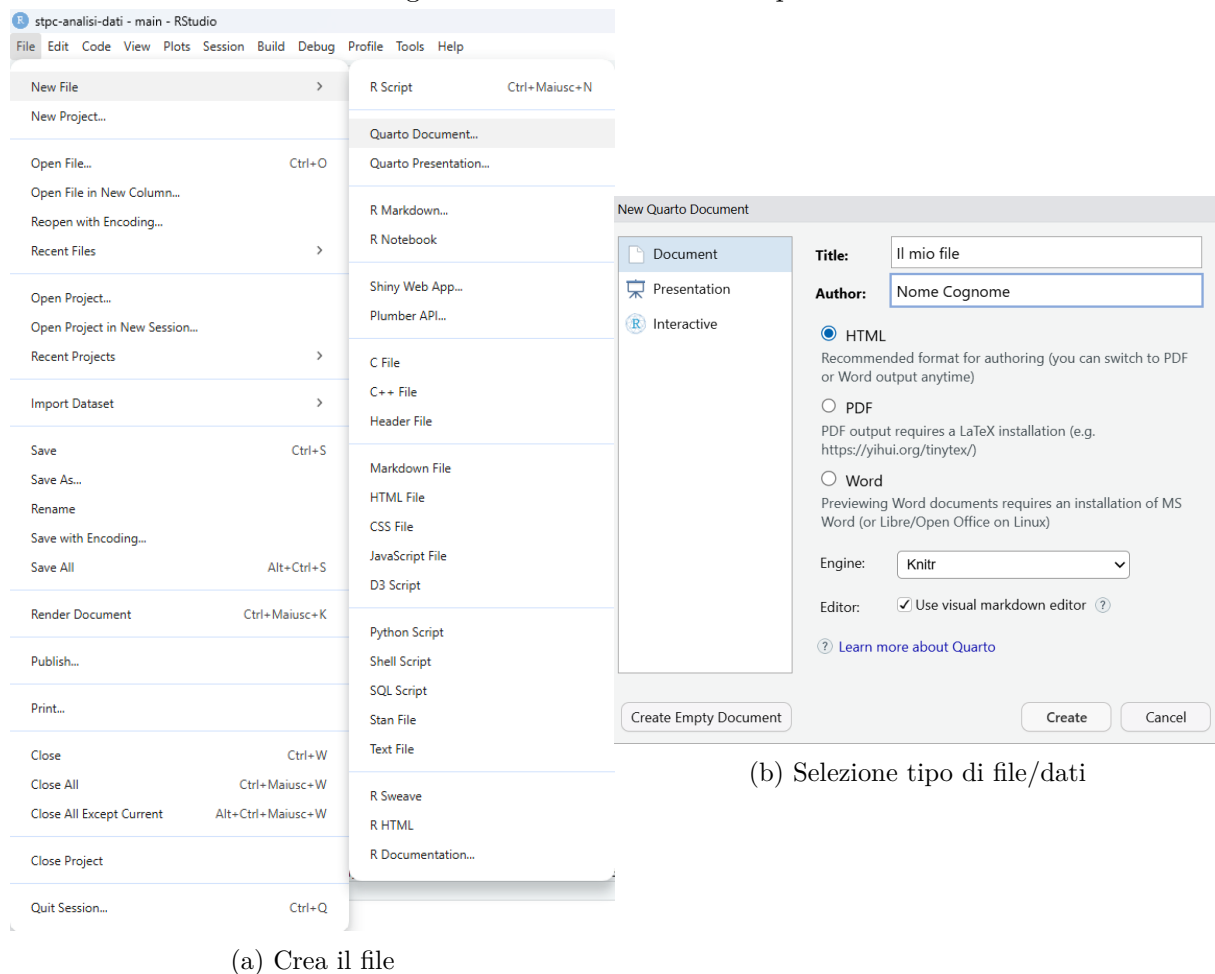


Figure 1.5 vi mostra il default che si ottiene quando si crea un documento quarto.

In verde in Figure 1.5, lo YAML che controlla l'aspetto generale del documento. Va modificato per cambiare i metadati del documento.

In rosso in Figure 1.5, un chunk di codice eseguibile, ovvero una parte del documento che funziona come un qualsiasi script di R e in quanto tale permette di eseguire codice.

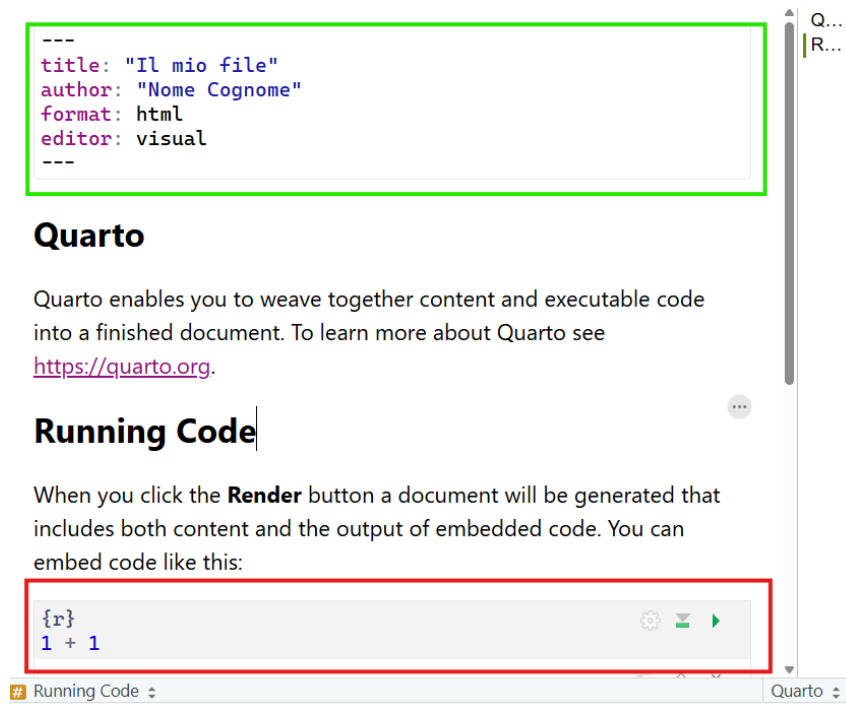


Figure 1.5: Default documento di quarto. In verde lo YAML, in rosso un chunk di codice eseguibile come qualsiasi script R

Modifica importante allo YAML

La parte `embed-resources: true` è fondamentale (e va scritta esattamente seguendo l'indentazione riportata). Serve a fare in modo che tutti i codici, i risultati e i grafici prodotti nel documento rimangano nel documento quando lo compilate in html.

```
---
title: "Il mio file"
author: "Nome Cognome"
format:
  html:
    embed-resources: true
editor: visual
---
```

Il chunk di codice funziona esattamente come R – è sequenziale, va seguito l'ordine con cui vengono fatte le operazioni

Ogni volta che viene fatto il rendering del documento (`shift + ctrl + k` o freccina azzurra), viene creato un ambiente temporaneo dove viene eseguito il codice.

L'ambiente temporaneo è indipendente dall'ambiente che vedete su R!

Per creare un nuovo chunk: `shift + ctrl + i` oppure Figure 1.6:

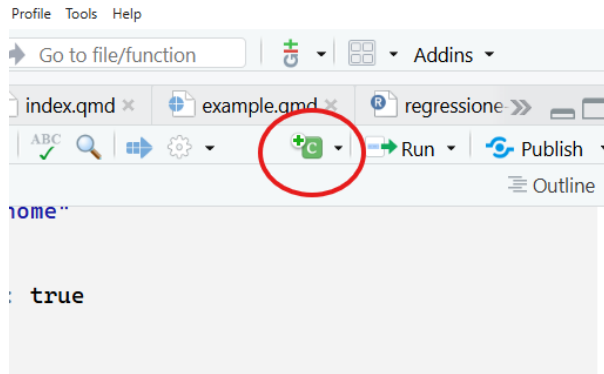


Figure 1.6: Creare un nuovo chunk di codice

1.5 Elementi base

1.5.1 Calcolatrice

```
3 + 2 # più
3 - 2 # meno
3 * 2 # per
3 / 2 # diviso
```

Parentesi e simili:

() [] {} " " : ; ,

ATTENZIONE

Nei chunk di codice che seguono vedrete spesso l'operatore `;` scritto sulla stessa riga di codice dove viene eseguita un'altra operazione, come ad esempio:

```
variabile = 1:5 ; variabile
```

```
[1] 1 2 3 4 5
```

equivale a scrivere

```
variabile = 1:5
variabile
```

```
[1] 1 2 3 4 5
```

1.5.2 Assign (Crea oggetti)

Tutto quello che viene creato in R viene chiamato *oggetto* tramite l'“assegnazione” con specifici simboli, ovvero `<-` oppure `=`:

```
x <- 4
x
```

```
[1] 4
```

```
X = 5
X
```

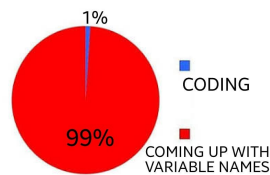
```
[1] 5
```

Gli elementi a destra del simbolo (contenuto) vengono assegnati all'elemento a sinistra (contenitore) del simbolo

Attenzione!

R è case sensitive. `x` e `X` sono due oggetti diversi!

1.5.3 Nominare gli oggetti



Nomi validi sono lettere, numeri, punti, underscore (e.g., `variable_name`)

Buoni nomi per le variabili:

- `mean_rt`
- `sum_age`
- `prop_item`

Pessimi nomi per variabili:

- `x`
- `y`
- `ciccio`

Non deve contenere caratteri speciali come `#`, `&`, `$`, `?`, etc.

Non deve essere una parola riservata ovvero quelle parole che sono utilizzate da R con un significato speciale (`?reserved` nella console).

```
.01 = 5
```

```
Error in 0.01 = 5: membro di sinistra dell'assegnazione (do_set) non valido
```

```
1 = "a"
```

Error in 1 = "a": membro di sinistra dell'assegnazione (do_set) non valido

```
TRUE = 567
```

Error in TRUE = 567: membro di sinistra dell'assegnazione (do_set) non valido

1.5.3.1 Per convenzione

Queste istruzioni sono valide attraverso diversi linguaggi di programmazione.

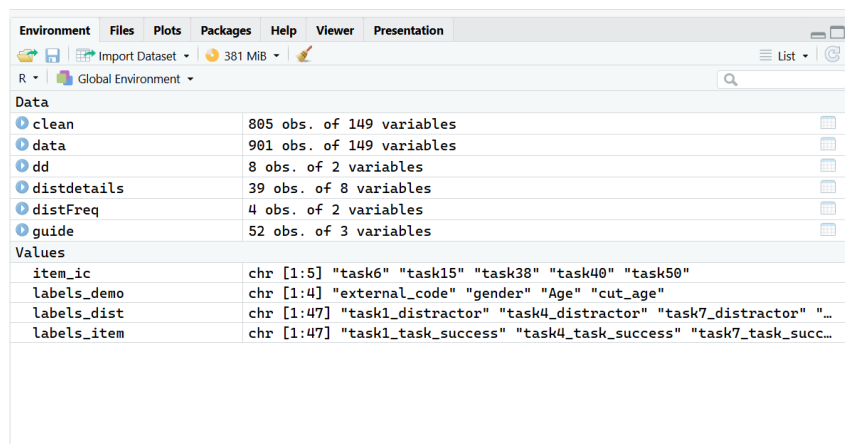
Fanno riferimento a nomi di oggetti composti da più nomi e/o molto lunghi:

1. `first_data`, ovvero separare i termini con un underscore (e.g., `my_data`)
2. `firstData`, ovvero la prima parola tutta minuscola, la seconda con la prima lettera maiuscola (e.g., `unitnData` invece di `university_of_trento_data`)

1.5.4 L'ambiente

Gli oggetti finiscono nell'ambiente (`environment`) di R.

La funzione `ls()` permette di vedere tutti gli oggetti all'interno dell'ambiente.



Environment	
Files	Plots
Packages	Help
Viewer	Presentation
R - Global Environment	
Data	
clean	805 obs. of 149 variables
data	901 obs. of 149 variables
dd	8 obs. of 2 variables
distdetails	39 obs. of 8 variables
distFreq	4 obs. of 2 variables
guide	52 obs. of 3 variables
Values	
item_ic	chr [1:5] "task6" "task15" "task38" "task40" "task50"
labels_demo	chr [1:4] "external_code" "gender" "Age" "cut_age"
labels_dist	chr [1:47] "task1_distractor" "task4_distractor" "task7_distractor" "..."
labels_item	chr [1:47] "task1_task_success" "task4_task_success" "task7_task_succ..."

Figure 1.7: Ambiente di R

`rm()` (remove) è la funzione preposta per rimuovere gli oggetti dall'ambiente di R: `rm("oggetto")` (vedi Listing 1.6).

Il codice `rm(list = ls())` permette di eliminare tutti gli oggetti nell'ambiente in un colpo solo (equivale alla scopetta).

Listing 1.4 Ottenere gli elementi nell'ambiente e uso della funzione `rm`

```
ls() # ottiene tutti gli oggetti nell'ambiente
```

Listing 1.5 Ottenere gli elementi nell'ambiente e uso della funzione rm

```
rm(variabile) # rimuove l'oggetto variabile  
ls()
```

```
[1] "variabile" "x"          "X"
```

```
[1] "x" "X"
```

Listing 1.6 Ottenere gli elementi nell'ambiente e uso della funzione rm

```
rm(list = ls()) # rimuove tutto l'ambiente  
ls()
```

```
character(0)
```

1.6 Pacchetti e Funzioni

1.6.1 Funzioni

Ogni cosa fatta in R di fatto è riassumibile nelle *operazioni* fatte su *oggetti*

Le operazioni sono le **funzioni**, che permettono di modificare gli oggetti esistenti e di creare di nuovi.

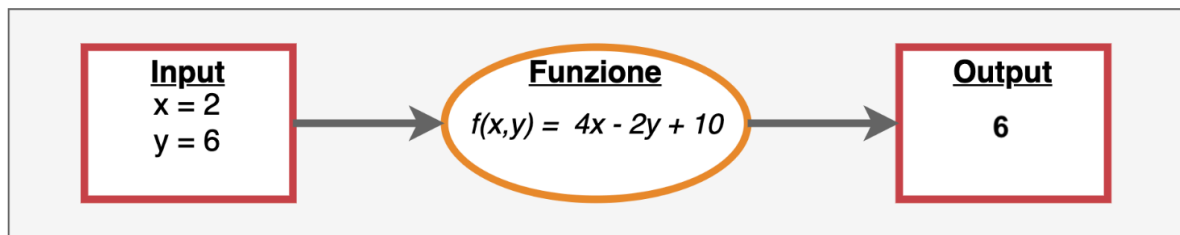
Ad esempio, in Listing 1.7 viene usata la funzione `sum()` per sommare tutti gli elementi all'interno del vettore. Matematicamente, la funzione `sum()` equivale a $\sum_{i=1}^k x_i$ dove k è il totale di elementi x all'interno del vettore X .

Listing 1.7 Applica la funzione somma per sommare tutti gli elementi all'interno di vettore

```
vettore = 1:5  
sum(vettore)
```

```
[1] 15
```

Le funzioni in R sono come le funzioni matematiche $f(\cdot)$, dove viene preso un input, viene elaborato secondo le regole della funzione e si ottiene un output:



1.6.1.1 Elementi di una funzione

```
mean(x, trim = 0, na.rm = FALSE, ...)
```

- Nome della funzione: `mean`
- Parentesi che racchiudono gli argomenti della funzione
- argomenti:
 - **obbligatori**, senza valori predefiniti senza di cui la funzione non può funzionare. Nel caso della funzione `mean()`, l'unico argomento obbligatorio è il vettore `x` di valori di cui si vuole calcolare la media
 - **facoltativi** con valori di default (`trim = 0`, `na.rm = FALSE`), che vengono applicati qualora l'utente non ne specifichi di alternativi. Nel caso della media, non viene applicato il trimming dei valori estremi e non vengono tolti i valori NA.

```
mean(vettore)
```

```
[1] 3
```

```
mean(vettore, trim = 0.3)
```

```
[1] 3
```

`?funzione` (Listing 1.8) permette di accedere all'help della funzione per scoprire esattamente cosa fa una funzione:

Listing 1.8 Permette di accedere all'help della funzione `mean`

```
?mean
```

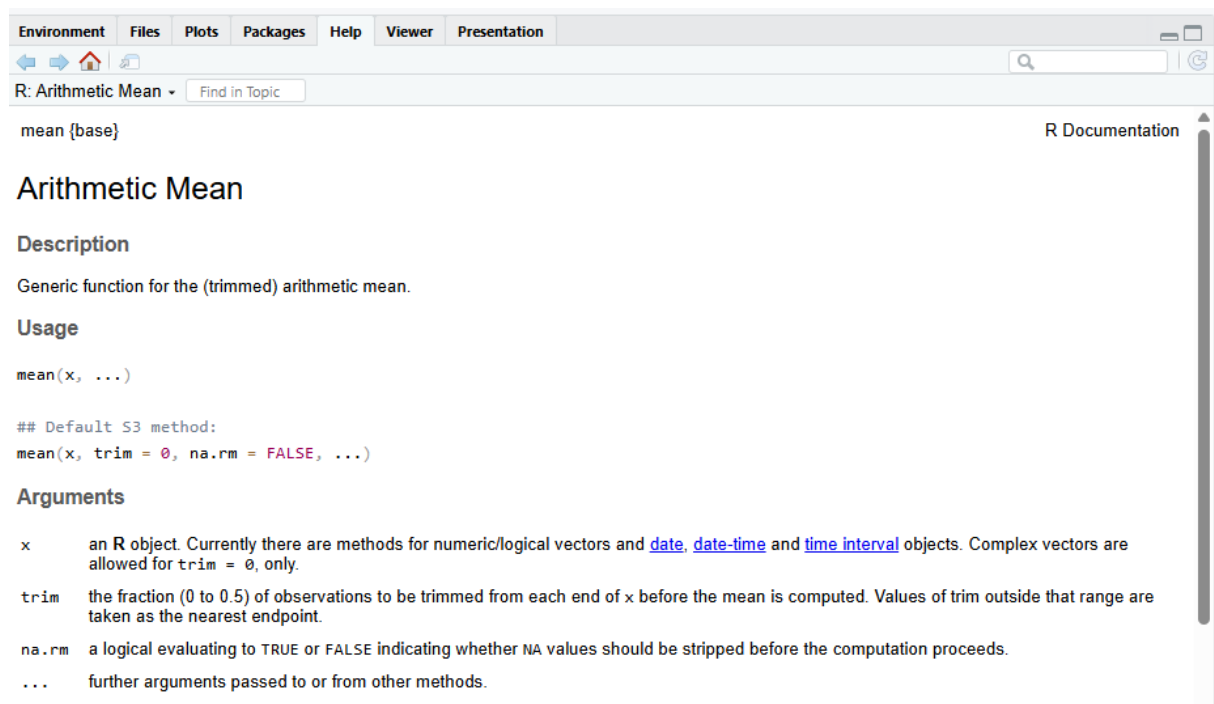


Figure 1.8: Help della funzione mean

1.6.1.2 Installare i pacchetti

Esistono funzioni predefinite all'installazione di R, se ne possono definire di nuove autonomamente, si possono installare dei “pacchetti” dal CRAN (comprehensive R archive network, vedi Listing 1.9) o da un file locale, a seconda delle operazioni che si vogliono svolgere.

Listing 1.9 Codice per installare e caricare un pacchetto

```
install.packages("nomePacchetto") # installa (va fatto correre solo una volta)
library(nomePacchetto) # carica il pacchetto (va fatto correre tutte le volte)
```

1.7 Operatori

1.7.1 Operatori Matematici

Tabella ?? illustra alcuni operatori matematici di base, mentre Tabella ?? illustra le principali funzioni matematiche disponibili in R.

1.7.2 Operatori Relazionali

In R è possibile valutare se una data relazione è vera o falsa. R valuterà le proposizioni e ci restituirà il valore **TRUE** se la proposizione è vera oppure **FALSE** se la proposizione è falsa. Per fare questo si usano gli operatori relazionali in Tabella ??.

Table 1.1: Operatori matematici in R

Funzione	Cosa fa?	Esempio	Risultato
+	addizione	5.4 + 6.1	11.5
-	sottrazione	9 - 4.3	4.7
*	moltiplicazione	7 * 1.4	9.8
/	divisione	9/3	3
%%	resto	9%%2	1
^	potenza	15 ^ 2	225

Table 1.2: Funzioni matematiche in R

Funzione	Cosa fa?	Esempio	Risultato
abs()	valore assoluto	abs(-8)	8
sqrt()	radice quadrata	sqrt(225)	15
exp()	funzione esponenziale	exp(0)	1
log()	logaritmo naturale	log(1)	0
round()	arrotondamento, intero	round(1.738)	2
round()	arrotondamento	round(1.738, 2)	1.74

Table 1.3: Operatori relazionali in R

Funzione	Nome	Esempio	Risultato
==	uguale	30 == 30	TRUE
!=	diverso	30 != 30	FALSE
> / >=	maggiore / maggiore o uguale	30 > 10 30 >= 10	TRUE TRUE
< / <=	minore / minore o uguale	30 < 10 10 <= 10	FALSE TRUE
%in%	inclusione	10 %in% c(1, 2, 10)	TRUE

Table 1.4: Operatori logici in R (con $x = 30$)

Funzione	Nome	Esempio	Risultato
<code>&</code>	Congiunzione	<code>x > 25 & x < 60</code>	TRUE
<code> </code>	Disgiunzione inclusiva	<code>x > 25 x > 60</code>	TRUE
<code>!</code>	Negazione	<code>!(x < 18)</code>	TRUE

Table 1.6: Tavole di verità

(a) Congiunzione

a	b	$a \cap b$
V	V	V
V	F	F
F	V	F
F	F	F

(b) Disgiunzione

a	b	$a \cup b$
V	V	V
V	F	V
F	V	V
F	F	F

(c) Negazione

a	$\neg a$
V	F
F	V

(d) Differenza Simmetrica

p	q	$p \oplus q$
V	V	F
V	F	V
F	V	V
F	F	F

1.7.3 Insiemistici

Per capire gli operatori insiemistici (Tabella 1.4), definiamo un oggetto, x , come segue:

```
x = 30
```

Gli operatori insiemistici permettono di testare diverse condizioni.

La **congiunzione** testa che siano contemporaneamente vere entrambe le condizioni, ovvero restituisce un risultato vero se e solo se entrambe le condizioni sono vere.

La **disgiunzione** testa che sia vera almeno una delle condizioni, ovvero restituisce un risultato vero se almeno una delle condizioni è vera.

La **negazione** inverte il valore logico.

1.8 Tavole di verità

Operazione	Simbolo	In R	Significato
Congiunzione (AND)	$A \cap B$	<code>&</code>	Vera se entrambe le condizioni sono vere Table 1.6a.
Disgiunzione (OR)	$A \cup B$	<code> </code>	Vera se almeno una delle due condizioni è vera Table 1.6b.
Negazione (NOT)	$\bar{A}$	<code>!</code>	Inverte il valore logico di A Table 1.6c.
Differenza simmetrica (XOR)	$A \Delta B$	<code>xor(A, B)</code>	Vera se una e una sola delle due condizioni è vera Table 1.6d.

1.9 R ed errori

Ci sono diversi livelli di **allerta** quando scriviamo codice:

- **messaggi**: la funzione ci restituisce qualcosa che è utile sapere, ma tutto liscio. Appaiono spesso quando si caricano dei pacchetti.
- **warnings**: la funzione ci informa di qualcosa di *potenzialmente* problematico, ma (circa) tutto liscio. R ci sta dicendo che è successo qualcosa di insolito e di prestare attenzione, ma non per forza è un problema.
- **error**: la funzione non solo ci informa di un **errore** ma le operazioni richieste non sono state eseguite. Solitamente il messaggio di errore ci dice anche cosa sta succedendo.

Come risolvere?

- Capire il messaggio
- Leggere la documentazione della funzione
- Cercare il messaggio su internet
- Chiedere aiuto nei forum dedicati
- Chiedere all'IA

1.9.0.1 Esempi di errori

```
mean(myVector)  #
```

Error: oggetto 'myVector' non trovato

```
dati = read.csv("i-miei-dati.csv")
```

Warning in file(file, "rt"): non è possibile aprire il file 'i-miei-dati.csv':
No such file or directory

Error in file(file, "rt"): non è possibile aprire la connessione

```
lenght(vettore)
```

Error in lenght(vettore): non trovo la funzione "lenght"

```
(3,4,5)
```

Error in parse(text = input): <text>:1:3: ', ' inatteso
1: (3,
 ^

1.9.0.2 R e l'uso dell'IA

I prompt devono più specifici possibile e centrati su quello che volete fare e come lo volete fare.

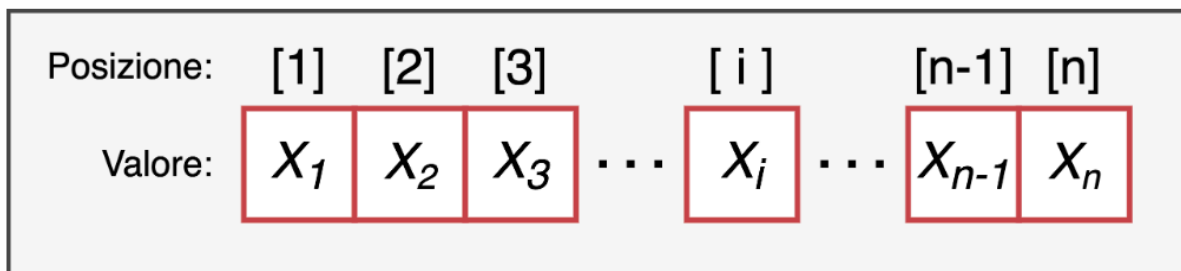
Dovete comunicare chiaramente che volete sempre la soluzione in R base, possibilmente senza usare funzioni aggiuntive e dovete sempre chiedere che vi spieghi il codice.

Warning

All'esame non potete usare l'IA, ma la potete usare per prepararvi e per aiutarvi nello studio

1.10 Vettori

I vettori sono una struttura dati unidimensionale e sono la più semplice presente in R.



Caratteristiche:

- **la lunghezza:** il numero di elementi da cui è formato il vettore. La funzione `length(x)` permette di ottenere la lunghezza del vettore `x`, ovvero permette di conoscere il numero di elementi di cui è composto il vettore.
- **la tipologia:** la tipologia di dati da cui è formato il vettore. Un vettore infatti deve essere formato da **elementi tutti dello stesso tipo** (vedi Section [1.10.2](#))!

I vettori sono caratterizzati da:

- **un valore:** il valore dell'elemento che può essere di qualsiasi tipo ad esempio un numero o una serie di caratteri.
- **un indice di posizione:** un numero intero positivo che identifica la sua posizione all'interno del vettore.

1.10.1 Creare un vettore con `c()`

Concatena diversi oggetti insieme per creare un nuovo oggetto unico

```
x = c(1, 2, 3) ; x
```

```
[1] 1 2 3
```

```
X = 1:3 ; X
```

```
[1] 1 2 3
```

```
x == X
```

```
[1] TRUE TRUE TRUE
```

1.10.2 Tipologie di vettore

Tipi di oggetti diversi → Tipi di vettori diversi:

- `num` (`numeric`): Numeri (-0.56, 3, 4.5)
- `logi` (`logic`): Logico (TRUE vs FALSE)
- `chr` (`character`): characters ("a", "b", "c")
- `factor` (`factor`): factor con diversi livelli

La distinzione più importante è tra vettori di tipo categoriale (`chr` e `factor`) e vettori di tipo numerico (`int` e `num`):

- `integer` e `numeric`: Operazioni matematiche E calcolo delle frequenze
- `character` e `factor`: Calcolo delle frequenze

```
months = c(5, 6, 8, 10, 12, 16); months
```

```
[1] 5 6 8 10 12 16
```

```
weight = runif(6, 3, 18) ; weight
```

```
[1] 8.650683 12.622100 13.940390 5.746200 5.569126 9.143152
```

I vettori logici si ottengono testando delle condizioni sulla base dei valori di altri vettori (Listing 1.10):

Listing 1.10 Testa se ci sono valori maggior di 12 all'interno del vettore

```
months > 12
```

```
[1] FALSE FALSE FALSE FALSE FALSE TRUE
```

Il vettore logico risultante può essere utilizzato per filtrare i valori dal vettore originale (vedi Section 1.10.3).

La funzione `rep()` in Listing 1.11 crea un vettore di tipo `character` ripetendo i K valori per X volte. Nell'esempio in Listing 1.11, le categorie `low`, `medium`, `high` vengono ripetute tutte per due volte (argomento `each = 2`).

La funzione `factor()` forza il vettore ad essere un vettore con K categorie e un numero di valori uguale al numero di elementi dentro il vettore.

```
[1] low    low    medium medium high   high
Levels: high low medium
```

L'argomento `levels` all'interno della funzione `factor()` (vedi Listing 1.12) permette di specificare l'ordine delle categorie, altrimenti il default di R le mette in ordine alfabetico.

Listing 1.11 Creare un vettore di tipo factor usando la funzione rep

```
ses = factor(rep(c("low", "medium", "high"), each = 2))
ses
```

Listing 1.12 Ordinare i livelli della variabile con levels

```
ses = factor(ses, levels = c("low", "medium", "high"))
```

1.10.3 Indicizzare i vettori

I vettori si indicizzano usando le parentesi quadre:

`nome_vettore[indice]` dove l'indice è la posizione all'interno del vettore che si vuole ottenere.

In R, la numerazione delle posizioni all'interno del vettore va da 1 alla lunghezza del vettore.

Dato ad esempio il vettore `nomi` come segue

```
nomi = c("Pasquale", "Egidio", "Debora", "Luca", "Andrea")
nomi
```

```
[1] "Pasquale" "Egidio"    "Debora"    "Luca"      "Andrea"
```

Possiamo immaginare ogni nome come occupante di una specifica posizione all'interno del vettore, come segue:

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

Per cui possiamo estrarre i vari nomi facendo riferimento al loro indice:

```
nomi[1] → Pasquale
```

```
nomi[3] → Debora
```

```
nomi[seq(2, 5, by = 2)] → Egidio, Luca
```

```
seq(min, max, by)
```

Questa funzione genera un vettore partendo da un valore minimo fino a un valore massimo con uno step incrementale definito dall'utente:

```
seq(1,5, by = 0.5)
```

```
[1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

L'indicizzazione dei vettori funziona anche su vettori di tipo numerico, come negli esempi seguenti:

```
weight[2] # secondo elemento di weight
```

```
[1] 12.6221
```

```
(weight[6] = 15.2) # sostituisce il sesto elemento di weight
```

```
[1] 15.2
```

```
weight[seq(1, 6, by = 2)] # elementi 1, 3, 5
```

```
[1] 8.650683 13.940390 5.569126
```

```
weight[2:6] # elementi dal 2 al 6 (incluso)
```

```
[1] 12.622100 13.940390 5.746200 5.569126 15.200000
```

```
weight[-2] # rimuove il secondo elemento
```

```
[1] 8.650683 13.940390 5.746200 5.569126 15.200000
```

I vettori logici possono essere utilizzati per estrarre determinati valori da un vettore:

```
weight > 7
```

```
[1] TRUE TRUE TRUE FALSE FALSE TRUE
```

Restituisce un vettore logico dove si TRUE se il valore in quella posizione è maggiore di 7 e FALSE in ogni altro caso. In questo caso si hanno valori veri (quindi osservazioni con un valore maggiore di 7) nelle seguenti posizioni del vettore:

```
[1] 1 2 3 6
```

Usando il vettore logico dentro le parentesi quadre, si otterranno esattamente i valori che occupano quelle posizioni:

```
weight[weight > 7] # solo i valori > di 7
```

```
[1] 8.650683 12.622100 13.940390 15.200000
```

```
weight[weight >= 4.5 & weight < 8] # valori compresi tra 4.5 e 8
```

```
[1] 5.746200 5.569126
```

1.11 Matrici

Le matrici sono una struttura dati **bidimensionale** (caratterizzate da 2 dimensioni, la funzione `dim()` permette di sapere da quante righe e da quante colonne è composta) dove il numero di righe rappresenta la dimensione 1 e il numero di colonne la dimensione 2.

```
my_mat = matrix(data = 1:10, nrow = 2, ncol = 5)
my_mat
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,]     1     3     5     7     9
[2,]     2     4     6     8    10
```

Per creare una matrice in R si può usare la funzione `matrix()`, come segue

```
`matrix(data, nrow, ncol, byrow = FALSE)`
```

dove `data` sono i valori che si vogliono mettere all'interno della matrice, `nrow` e `ncol` specificano il numero di righe e di colonne che si vogliono mettere nella matrice e `byrow = FALSE` (default) indica che i dati vengono organizzati per colonna (ovvero la matrice viene riempita seguendo le colonne).

```
# crea una matrice con 3 righe e 4 colonne
# (i valori sono sistemati per riga)
A = matrix(1:12, nrow=3, ncol = 4, byrow = TRUE)
# assegna dei nomi alle righe
rownames(A) = paste("a",1:nrow(A), sep = "_")
# assegna dei nomi alle colonne
colnames(A) = paste("b",1:ncol(A), sep = "_")
A
```

```
      b_1 b_2 b_3 b_4
a_1     1     2     3     4
a_2     5     6     7     8
a_3     9    10    11    12
```

Nel caso precedente, la lunghezza del vettore organizzato nella matrice rientrava perfettamente nel numero di celle disponibili all'interno della matrice stessa. Se questo non succede, si hanno due casi:

Il numero di celle è un multiplo della lunghezza del vettore

```
data = 1:12
matrix(data, nrow = 6, ncol =4)
```

```
      [,1] [,2] [,3] [,4]
[1,]     1     7     1     7
[2,]     2     8     2     8
```

```
[3,] 3 9 3 9
[4,] 4 10 4 10
[5,] 5 11 5 11
[6,] 6 12 6 12
```

Non succede niente, non appaiono warning e la matrice viene creata ripetendo due volte il vettore originale.

Il numero di celle NON è un moltiplicando della lunghezza del vettore

```
data = 1:12
matrix(data, nrow = 3, ncol = 2)
```

Warning in matrix(data, nrow = 3, ncol = 2): data length differs from size of matrix: [12 != 3 x 2]

```
      [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

Si ottiene un warning, ma R comunque crea la matrice come può.

1.11.1 Caratteristiche

Le matrici di fatto sono la concatenazione bidimensionale di una serie di vettori. Di conseguenza:

- Possono contenere una sola tipologia di dati
- Essendo bidimensionali, sono necessari due indici di posizione (righe e colonne) per identificare un elemento
- Possono essere viste come un insieme di singoli vettori

1.11.2 Indicizzare le matrici

La logica per l'indicizzazione è la stessa che si usa per i vettori, solo che all'interno delle parentesi quadre va inserito anche l'indice per le colonne:

```
my_matrix[righe, colonne]
```

Il primo indice fa sempre riferimento alle righe, il secondo alle colonne:

Table 1.7: Struttura di una matrice

	[,1]	[,2]	[,3]
[1,]	[1, 1]	[1, 2]	[1, 3]
[2,]	[2, 1]	[2, 2]	[2, 3]
[3,]	[3, 1]	[3, 2]	[3, 3]

Ad esempio, data la matrice A:

A

	b_1	b_2	b_3	b_4
a_1	1	2	3	4
a_2	5	6	7	8
a_3	9	10	11	12

`A[1, ]` → 1, 2, 3, 4

`A[, 2]` → 2, 6, 10

`A[2, 3]` → 7

1.12 Dataframe

Il dataframe è la struttura più “complessa”, utile e potente di R:

Come le matrici, è composto da due dimensioni (righe × colonne)

A differenza delle matrici, può contenere dati di tipo diverso

- Ogni elemento (colonna) è un vettore con un nome associato
- Ogni colonna deve avere lo stesso numero di elementi di conseguenza ogni riga ha lo stesso numero di elementi (struttura rettangolare)
- **Tutte le colonne (quindi tutti i vettori contenuti nel dataframe) devono avere la stessa lunghezza**

`data.frame(. . .)`

```
sexo = c(rep(c("M","F"), length=length(weight)))
data = data.frame(id = paste0("id", 1:length(weight)),
                  weight, sexo)
data
```

	id	weight	sexo
1	id1	8.650683	M
2	id2	12.622100	F
3	id3	13.940390	M
4	id4	5.746200	F
5	id5	5.569126	M
6	id6	15.200000	F

1.12.1 Indicizzare i dataframe

```
data[r, c], data$nome_variabile
```

```
data
```

```
   id    weight sesso
1 id1  8.650683     M
2 id2 12.622100     F
3 id3 13.940390     M
4 id4  5.746200     F
5 id5  5.569126     M
6 id6 15.200000     F
```

```
data[2, 1]
```

```
[1] "id2"
```

```
data$weight
```

```
[1]  8.650683 12.622100 13.940390  5.746200  5.569126 15.200000
```

```
data$weight[2]
```

```
[1] 12.6221
```

La stessa logica con cui dai vettori si estraggono tutte le osservazioni che soddisfano un determinato requisito

```
weight[weight >= 7]
```

```
[1]  8.650683 12.622100 13.940390 15.200000
```

Nel caso del dataframe:

```
data[data$weight >=7, ]
```

```
   id    weight sesso
1 id1  8.650683     M
2 id2 12.622100     F
3 id3 13.940390     M
6 id6 15.200000     F
```

```
data[data$weight >=7 & data$weight < 9,
      "id"]
```

```
[1] "id1"
```

1.12.2 Attributi e caratteristiche

Le funzioni `names()`, `dim()`, `nrow()`, `ncol()` sono alcune delle funzioni per ottenere informazioni sulle caratteristiche del dataframe.

La funzione più utile è `str()` poichè ci restituisce una veloce overview della struttura del dataframe: dimensioni, tipi di variabili,...

```
str(data)
```

```
'data.frame':  6 obs. of  3 variables:
 $ id      : chr  "id1" "id2" "id3" "id4" ...
 $ weight: num   8.65 12.62 13.94 5.75 5.57 ...
 $ sesso  : chr   "M" "F" "M" "F" ...
```

NB

La funzione `str()` funziona su tutti gli oggetti di R

1.13 Liste

```
list(name1 = obj1, name2 = obj2, name3 = obj3, . . .)
```

Permette di creare liste di oggetti diversi (matrici, vettori, variabili) di diverso tipo (`numeric`, `character`, `integer`)

Molti dei risultati ottenuti con le funzioni per le analisi statistiche sono organizzati in liste, proprio perché permettono di contenere oggetti di natura diversa e con dimensionalità diversa

```
A <- c(1,8,5,0)
M <- matrix(A, 2, 2)
Nomi3 <- c("giorgio", "ugo", "anna")
LV <- c(TRUE, FALSE, TRUE, TRUE)
OgLista <- list(a = A, Mat = M, nomi = Nomi3, X = LV)
str(OgLista)
```

```
List of 4
 $ a    : num [1:4] 1 8 5 0
 $ Mat  : num [1:2, 1:2] 1 8 5 0
 $ nomi : chr [1:3] "giorgio" "ugo" "anna"
 $ X    : logi [1:4] TRUE FALSE TRUE TRUE
```

1.13.1 Indicizzare le liste

```
lista[[i]], lista$nome_oggettpo
```

`[[i]]`: indice dell'oggetto all'interno della lista a cui si vuole accedere

```
OgLista[[2]]
```

```
      [,1] [,2]  
[1,]     1     5  
[2,]     8     0
```

```
$nome_oggetto
```

```
OgLista$Mat
```

```
      [,1] [,2]  
[1,]     1     5  
[2,]     8     0
```

1.14 Importare dati esterni

1.14.1 Importa base

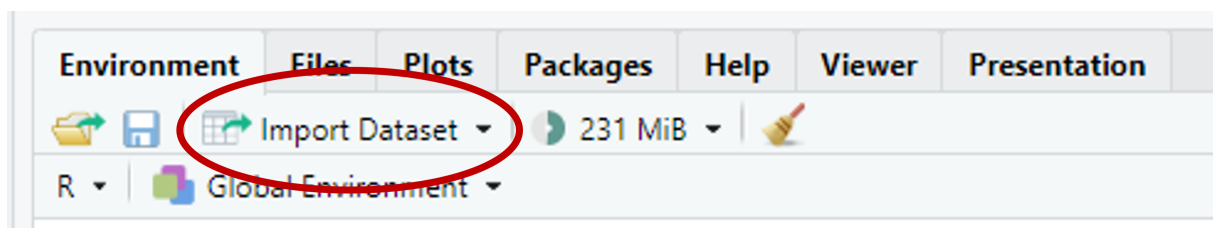
```
read.csv2("file_path", header = T, sep = ";")
```

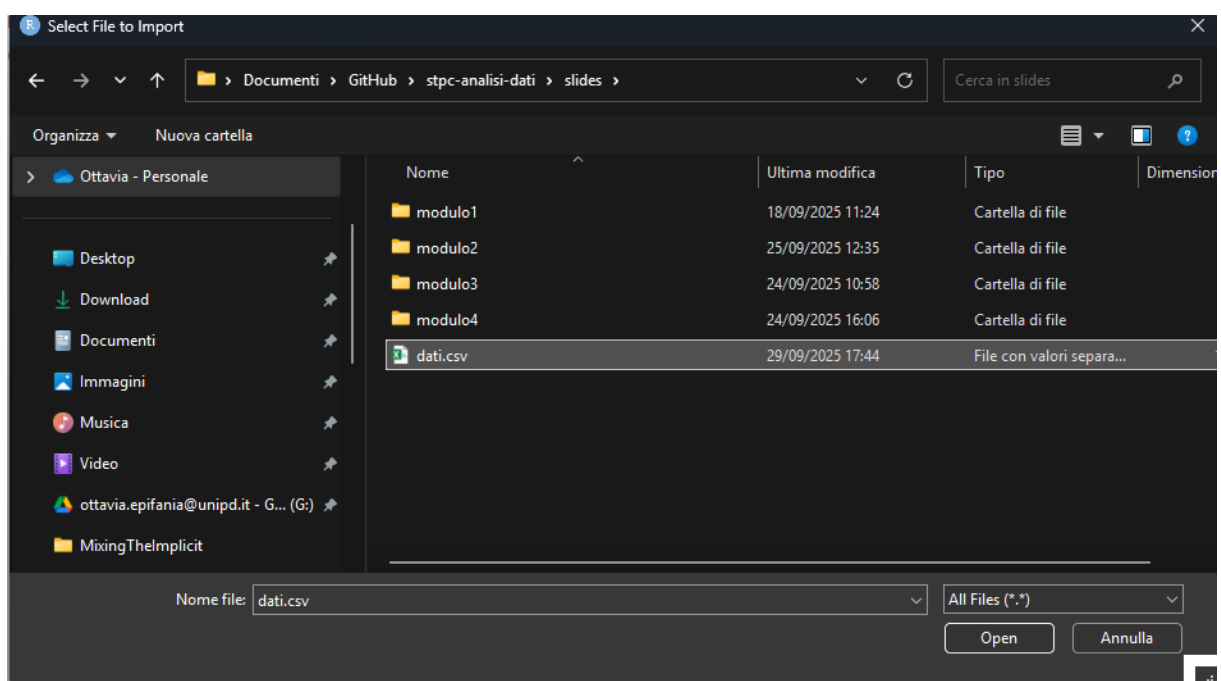
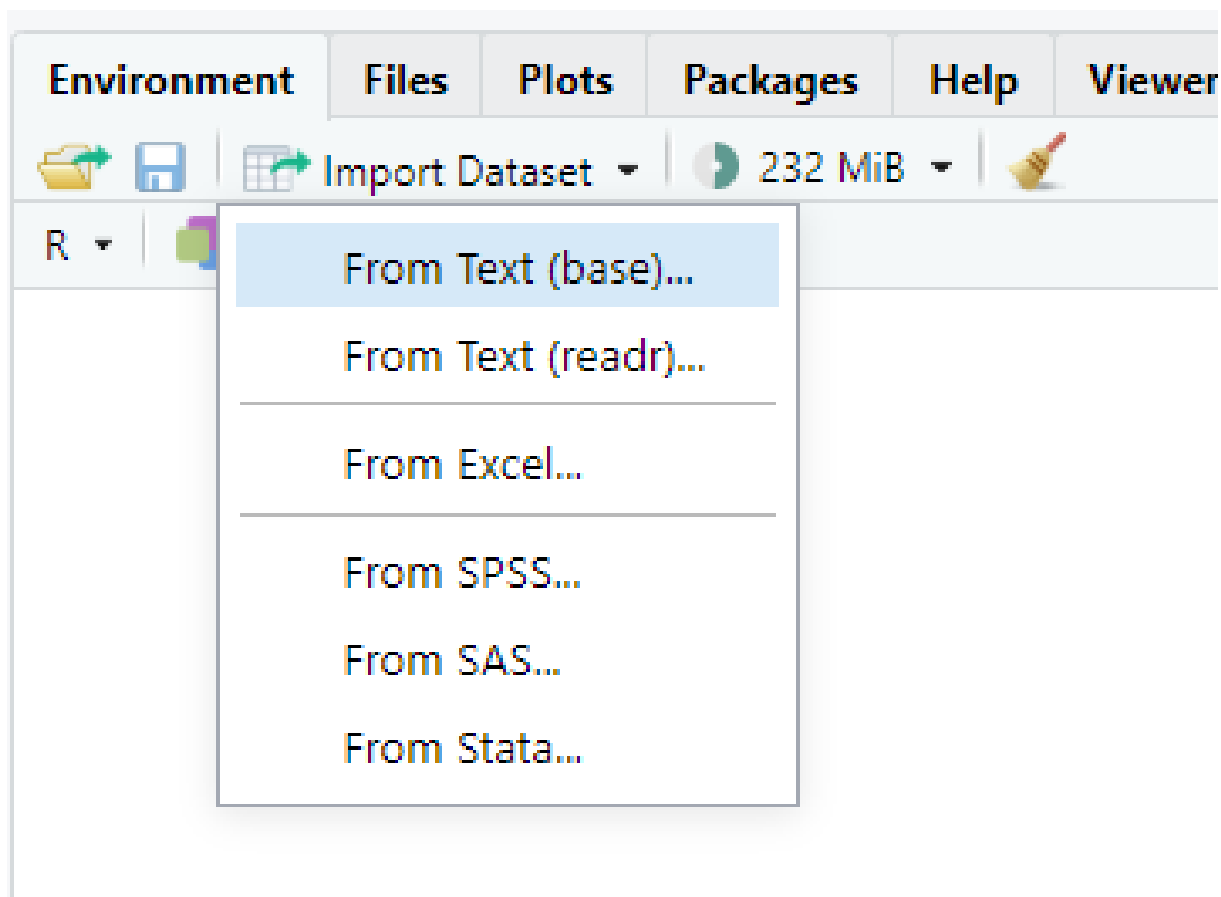
Il default della funzione prevede che la prima riga contenga i nomi delle variabili e che il separatore di colonne sia “;”.

Se si possiede un dataset chiamato “dati” nella cartella “data”, questo codice lo importerà all’interno dell’ambiente

```
# Lettura file dati esterni  
data <- read.csv2("data/dati.csv")
```

1.14.2 Importa alternativa 1





Import Dataset

Name:

Input File:
 sb1;18;M;nord
 sb2;19;F;nord
 sb3;21;F;nord
 sb4;18;F;nord
 sb5;19;M;centro
 sb6;22;M;sud
 sb7;18;M;centro
 sb8;20;F;sud
 sb9;18;F;nord
 sb10;19;F;centro
 sb11;20;M;nord
 sb12;18;M;sud
 sb13;18;M;sud
 sb14;19;F;centro
 sb15;21;F;centro

Encoding:

Heading: ☒ Yes ☐ No

Row names:

Separator:

Decimal:

Quote:

Comment:

na.strings:

☐ Strings as factors

Data Frame

ID	eta	Sesso	regione
sb1	18	M	nord
sb2	19	F	nord
sb3	21	F	nord
sb4	18	F	nord
sb5	19	M	centro
sb6	22	M	sud
sb7	18	M	centro
sb8	20	F	sud
sb9	18	F	nord
sb10	19	F	centro
sb11	20	M	nord
sb12	18	M	sud
sb13	18	M	sud
sb14	19	F	centro
sb15	21	F	centro

Import Cancel

1.14.3 Importa alternativa 2

```
dati = read.csv("slides/dati.csv",
               sep=";", dec=".", header = TRUE)
head(dati)
```

```
  ID eta sesso regione
1 sb1  18     M   nord
2 sb2  19     F   nord
3 sb3  21     F   nord
4 sb4  18     F   nord
5 sb5  19     M centro
6 sb6  22     M   sud
```

1.15 Sintesi delle strutture tabellari

1.15.0.1 summary()

```
summary(data)
```

Riassume il contenuto di strutture di dati

```
summary(rock)
```

area	peri	shape	perm
Min. : 1016	Min. : 308.6	Min. : 0.09033	Min. : 6.30
1st Qu.: 5305	1st Qu.: 1414.9	1st Qu.: 0.16226	1st Qu.: 76.45
Median : 7487	Median : 2536.2	Median : 0.19886	Median : 130.50
Mean : 7188	Mean : 2682.2	Mean : 0.21811	Mean : 415.45
3rd Qu.: 8870	3rd Qu.: 3989.5	3rd Qu.: 0.26267	3rd Qu.: 777.50
Max. : 12212	Max. : 4864.2	Max. : 0.46413	Max. : 1300.00

1.15.0.2 str()

```
str(data)
```

Fornisce informazioni circa la struttura generale di un oggetto:

```
str(rock)
```

```
'data.frame': 48 obs. of 4 variables:
 $ area : int 4990 7002 7558 7352 7943 7979 9333 8209 8393 6425 ...
 $ peri : num 2792 3893 3931 3869 3949 ...
 $ shape: num 0.0903 0.1486 0.1833 0.1171 0.1224 ...
 $ perm : num 6.3 6.3 6.3 6.3 17.1 17.1 17.1 17.1 119 119 ...
```

1.15.0.3 head() (α) and tail() (ω)

```
head()
```

Le prime 6 osservazioni del dataset

```
head(rock)
```

	area	peri	shape	perm
1	4990	2791.90	0.0903296	6.3
2	7002	3892.60	0.1486220	6.3
3	7558	3930.66	0.1833120	6.3
4	7352	3869.32	0.1170630	6.3
5	7943	3948.54	0.1224170	17.1
6	7979	4010.15	0.1670450	17.1

```
tail()
```

le ultime 6 osservazioni del dataset

```
tail(rock)
```

	area	peri	shape	perm
43	5605	1145.690	0.464125	1300
44	8793	2280.490	0.420477	1300
45	3475	1174.110	0.200744	580
46	1651	597.808	0.262651	580
47	5514	1455.880	0.182453	580
48	9718	1485.580	0.200447	580

1.15.0.4 apply()

`apply(data, MARGIN, FUN)`

data: i dati su cui si vuole lavorare

MARGIN: 1 applica la funzione sulle righe (attraverso le colonne), 2 applica la funzione sulle colonne (attraverso le righe)

FUN: la funzione che si vuole applicare

```
apply(rock, 1, sum) # Somma per riga
```

```
[1] 7788.290 10901.049 11495.143 11227.737 11908.762 12006.417 13696.040
[8] 12571.014 12194.244 9642.812 13963.201 12729.388 14770.169 12468.672
[15] 13498.943 12585.623 15629.464 15589.483 16801.020 14405.155 11408.854
[22] 16371.431 17051.890 11893.602 9077.671 6312.529 10072.432 7571.553
[29] 9250.435 7025.001 9393.452 10245.336 8356.958 7258.715 9692.914
[36] 10305.336 4945.877 2044.761 4813.624 7012.214 7289.872 2624.872
[43] 8051.154 12373.910 5229.311 2829.071 7550.062 11783.780
```

```
apply(rock, 2, mean) # Media per colonna
```

	area	peri	shape	perm
	7187.7291667	2682.2119375	0.2181104	415.4500000

2 Modulo 2 – Scale di misura e statistica descrittiva univariata

2.1 Variabili

QUALITATIVE / CATEGORIALI

- sconnesse: $\{a, b, c\}$
- ordinate: $\{a, b, c\}$ con $a < b < c$ (oppure $a > b > c$)

QUANTITATIVE / METRICHE

- Discrete: $\mathbb{Z}$
- Continue: $\mathbb{R}$

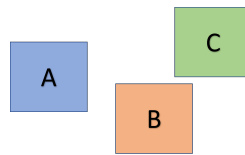
2.1.0.1 Stevens (1946)

Si differenziano in base alla quantità di informazione che può essere ricavata

- Variabili qualitative
 - **Nominale:** Distingue un insieme di dati in diverse categorie
 - **Ordinale:** Distingue un insieme di dati in diverse categorie che sono ordinabili a seconda della quantità di caratteristica posseduta
- Variabili quantitative
 - **Intervalli equivalenti** Distingue un insieme di dati in diverse categorie ordinabili. La distanza tra le categorie è nota perché c'è un'unità di misura
 - **A rapporti equivalenti:** Come la scala ad intervalli...ma lo 0 indica assenza di caratteristica (non è arbitrario ma è assoluto)

2.1.1 Categoricali - Nominali

Comune di residenza, sesso biologico, patologia clinica ecc. Vale solo l'equivalenza ($=$ o $\neq$)



Al posto di A, B, C si poteva scrivere α , β , γ , 1,2,3 (ma i numeri valgono solo come etichette!)

Caratteristiche:

- Categorie **distintive**: gli elementi che appartengono a categorie differenti vengono considerati di tipo diverso
- Categorie **mutualmente escludentesi**: ogni elemento può rientrare in una ed una sola categoria
- Categorie **collettivamente esaustive**: tutti gli elementi vengono classificati nelle categorie della variabile, nessuno escluso

2.1.1.1 `as.factor()`

`as.factor()`: Converte una variabile in una variabile di tipo **categoriale/nominale** con tanti livelli quanti sono i valori unici nel vettore

Due livelli

```
Genere <- c("m", "f", "f", "m", "f", "m")
Genere
```

```
[1] "m" "f" "f" "m" "f" "m"
```

```
as.factor(Genere)
```

```
[1] m f f m f m
Levels: f m
```

`as.character()` produce una variabile categoriale, ma senza i livelli che caratterizzano i factor

Più livelli

```
diagnosi <- c("schizo", "dep", "narc", "schizo", "dep", "narc")
diagnosi
```

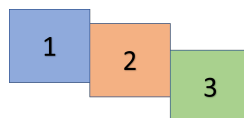
```
[1] "schizo" "dep"    "narc"   "schizo" "dep"    "narc"
```

```
as.factor(diagnosi)
```

```
[1] schizo dep    narc  schizo dep    narc  
Levels: dep narc schizo
```

2.1.2 Categoricali - Ordinali

Titolo di studio, gradimento di un prodotto, valutazione di una malattia (e.g., alto, medio, basso), fasce di reddito ecc. Vale sia l'equivalenza ($=$ o $\neq$) sia l'ordinamento ($<$ o $>$)



Attenzione!

Si conosce l'ordine tra le categorie, non si conosce la distanza.

Le etichette numeriche valgono per il loro ordine (non ha senso compiere operazioni tra di loro)

```
factor(A,levels=c("a1","a2",..., "ak"), ordered=TRUE)
```

Converte la variabile in una variabile con tanti livelli quanti sono i valori unici e viene forzato un ordine su questi livelli:

```
Voto <- c("alto","medio","medio","basso","alto")  
Voto
```

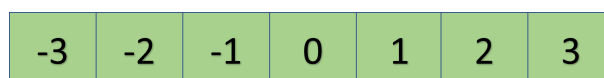
```
[1] "alto" "medio" "medio" "basso" "alto"
```

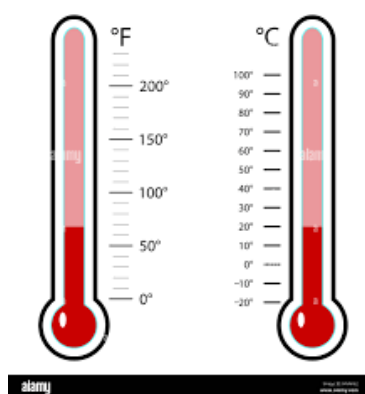
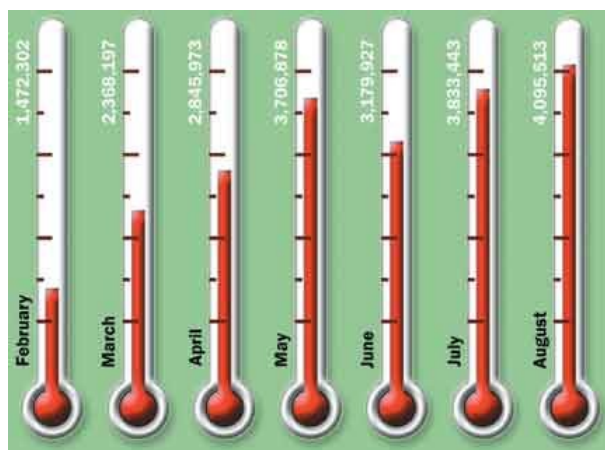
```
factor(Voto,levels=c("basso","medio","alto"), ordered=TRUE)
```

```
[1] alto medio medio basso alto  
Levels: basso < medio < alto
```

2.1.3 Quantitative – Scala ad intervalli equivalenti

Lo 0 è arbitrario, ma c'è un'*unità di misura* per cui le categorie si trovano alla stessa distanza. Temperatura in Celsius, Q.I., scale di atteggiamento ecc. Vale equivalenza ($=$ o $\neq$), ordinamento ($<$ o $>$) e differenza ($+$ o $-$):





Se ci sono 0°C , non vuol dire che non c'è temperatura. Se ieri c'erano 5°C e oggi ce ne sono 10°C , possiamo dire che oggi fa più caldo di ieri e che ci sono 5°C più di ieri. Il fatto che lo 0 non sia arbitrario non ci permette di dire che oggi c'è il doppio del caldo di ieri. Trasformando le due temperature in Fahrenheit si ottiene 41°F e 50°F ... e la seconda non è più il doppio della prima!

2.1.4 Quantitative – Scala a rapporti equivalenti

Lo 0 è assoluto, indica assenza del fenomeno. Peso, altezza, valori diagnostici. Vale equivalenza ($=$ o $\neq$), ordinamento ($<$ o $>$), differenza ($+$ o $-$) e rapporto ($\times$ o $\div$)

0	1	2	3
---	---	---	---



2.1.4.1 numeric, integer()

Le **variabili metriche** in R sono variabili di tipo **numeric** o **integer**, a seconda che siano continue o discrete

Attenzione!

In R non esiste una distinzione tra variabili a intervalli equivalenti e variabili a rapporti equivalenti – Sta a voi capire che tipo di variabile sia

```
x = seq(-3, 3, by = .5); x ①  
y = rnorm(10, mean = 3, sd=2); y ②  
z = 1:15; z ③
```

- ① La funzione `seq()` genera un vettore di numeri da un minimo a un massimo, con un step progressivo definito dall'utente
- ② La funzione `rnorm()` genera un vettore di lunghezza definita estraendo a caso da una distribuzione normale
- ③ L'operatore `:` genera una sequenza unitaria di valori da un minimo a un massimo

```
[1] -3.0 -2.5 -2.0 -1.5 -1.0 -0.5  0.0  0.5  1.0  1.5  2.0  2.5  3.0  
[1]  2.3775862  0.7097891  1.6381914  4.7637100  5.9610164  5.7959862  
[7]  2.4273062  5.2598693 -0.4288729  1.6312183  
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

```
str(x)
```

```
num [1:13] -3 -2.5 -2 -1.5 -1 -0.5 0 0.5 1 1.5 ...
```

```
str(y)
```

```
num [1:10] 2.38 0.71 1.64 4.76 5.96 ...
```

```
str(z)
```

```
int [1:15] 1 2 3 4 5 6 7 8 9 10 ...
```

2.2 Frequenze

2.2.1 Assolute e relative

Categoria x_i	n_i	$\%_i = \frac{n_i}{n} \cdot 100$
x_1	n_1	$\%_1$
x_2	n_2	$\%_2$
$\vdots$	$\vdots$	$\vdots$
x_k	n_k	$\%_k$
Totale	n	100

Frequenza assoluta: $n_i \geq 0$, con $n_i \in \mathbb{N}$ e $\sum_{i=1}^k n_i = n$

Frequenza percentuale: $\%_i = \frac{n_i}{n} \cdot 100$, con $0 \leq \%_i \leq 100$ e $\sum_{i=1}^k \%_i = 100$

```
table(variabile)
```

Calcola le frequenze per ogni categoria. Funziona sia per le variabili categoriali sia per le variabili metriche

```
table(Genere)
```

```
Genere
f m
3 3
```

```
table(diagnosi)
```

```
diagnosi
  dep  narc schizo
    2    2      2
```

2.2.2 Cumulate

Frequenza cumulata: numero o percentuale di unità statistiche che presentano una categoria “minore o uguale” alla categoria corrente.

Frequenze assolute cumulate

$$N_i = n_1 + n_2 + \dots + n_i = \sum_{j=1}^i n_j$$

con $N_1 = n_1$ e $N_k = n$.

Frequenze percentuali cumulate

$$N_i\% = n_1\% + n_2\% + \dots + n_i\% = \sum_{j=1}^i n_j\%$$

con $N_1\% = n_1\%$ e $N_k\% = 100\%$.

Le frequenze cumulate possono essere definite **ricorsivamente**:

$$N_i = N_{i-1} + n_i, \quad N_i\% = N_{i-1}\% + n_i\%$$

Attenzione

Le frequenze cumulate si calcolano **solo quando le modalità del carattere presentano un ordinamento**.

Non hanno quindi senso per caratteri qualitativi sconnessi (nominali).

Esprimi l'accordo con l'affermazione:

Insegnare statistica a psicologia è una pessima idea

Modalità x_i	n_i	N_i	$n_i\%$	$N_i\%$
Totalmente in disaccordo	10	10	24.4%	24.4%
Abbastanza in disaccordo	8	18	19.5%	43.9%
Indifferente	6	24	14.6%	58.5%
Abbastanza d'accordo	14	38	34.2%	92.7%
Totalmente d'accordo	3	41	7.3%	100.0%
Totale	41		100.0%	

2.2.2.1 cumsum()

```
opinione = data.frame(  
  categoria = c("Totalmente in disaccordo", "Abbastanza in disaccordo",  
    "Indifferente",  
    "Abbastanza d'accordo", "Totalmente d'accordo"),  
  n = c(10, 8, 6, 14, 3)  
)
```

```
opinione$N = cumsum(opinione$n)
opinione
```

	categoria	n	N
1	Totalmente in disaccordo	10	10
2	Abbastanza in disaccordo	8	18
3	Indifferente	6	24
4	Abbastanza d'accordo	14	38
5	Totalmente d'accordo	3	41

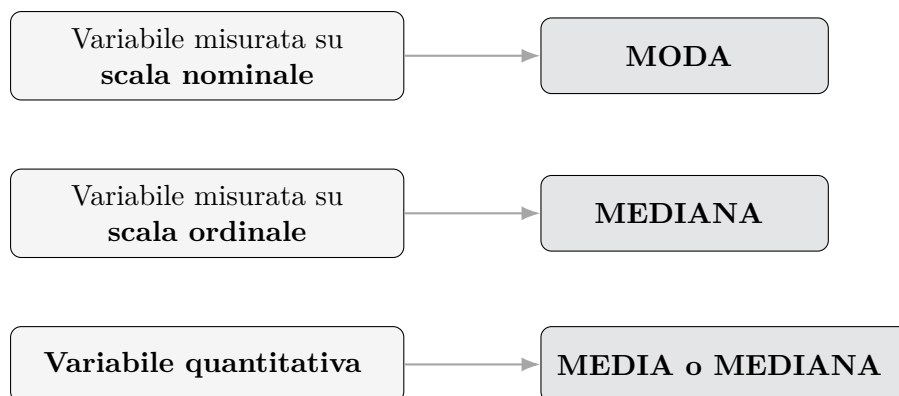
2.3 Indici di posizione

- **indici sintetici** che evidenziano le caratteristiche essenziali della distribuzione della variabile

Presentano livelli maggiori di machiavellismo i3 CEO o i3 impiegat3?

- Si confrontano categorie che rappresentano i livelli o valori tipici
- Consentono di sintetizzare un insieme di misure tramite un unico valore “rappresentativo” → indice che riassume o descrive i dati e dipende dalla scala di misura dei dati in oggetto
- moda
- mediana
- quartili/percentili
- media aritmetica

Ogni variabile statistica ha l'indice di posizione adeguato: non tutti gli indici si possono calcolare per ogni variabile → **indice di posizione adeguato**



2.3.0.1 Moda (Mo)

Categoria che presenta la massima frequenza

$$Mo(X) = \max(n_i)$$

(categoria con massimo valore di n_i)

Warning

N.B: Si può calcolare per le variabili misurate a qualsiasi livello, anche se di fatto viene calcolata solo per le variabili categoriali, in quanto per altri livelli di misurazione si possono calcolare altri indici più informativi

opinione

	categoria	n	N
1	Totalmente in disaccordo	10	10
2	Abbastanza in disaccordo	8	18
3	Indifferente	6	24
4	Abbastanza d'accordo	14	38
5	Totalmente d'accordo	3	41

```
max(opinione$n) # restituisce max(n_i)
```

```
[1] 14
```

```
opinione[opinione$n == max(opinione$n), "categoria"]
```

```
[1] "Abbastanza d'accordo"
```

2.3.0.2 Mediana

Categoria/valore che occupa la posizione centrale o mediana (Pos_{Me}) nella distribuzione ordinata dei dati

- preceduta da almeno 50% dei casi
- superata da almeno 50% dei casi

$$Pos_{Me} = \frac{(N + 1)}{2}$$

Il valore di Pos_{Me} va cercato nella distribuzione delle frequenze cumulate e si sceglie la categoria con $N_i \geq Pos_{Me}$

2.3.0.3 Quantili

Categorie/valori che dividono la distribuzione di frequenza ordinata in più parti.

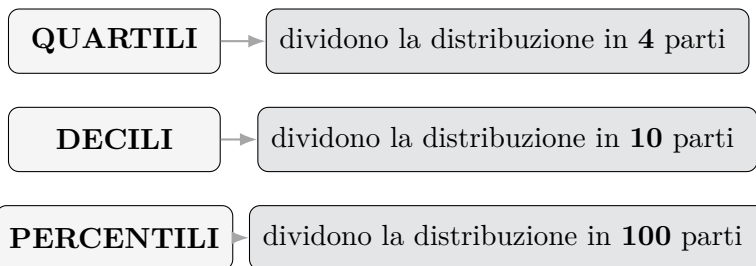
Qual è il reddito familiare che divide il 25% dei più poveri dal restante 75% ?

Qual è la soglia di reddito oltre cui sta la fascia dei più abbienti ?

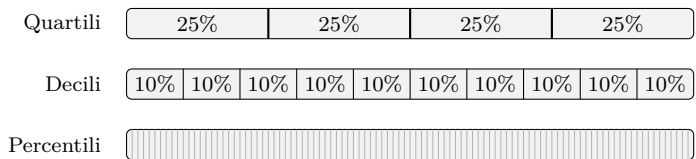
Quali sono le persone che hanno performato al meglio nell'ultimo semestre?

Il quantile x_p di ordine p è quella categoria/valore che è:

- preceduta da almeno $p\%$ dei casi
- superata da almeno $(1 - p)\%$ dei casi



La distribuzione è la stessa: cambia il modo in cui viene suddivisa



Per il calcolo dei quantili di diverso ordine, il procedimento è analogo a quello della Mediana, che di fatto è un caso particolare di questi indici. In generale la Posizione di ordine p (in centesimi) si ottiene:

$$Pos_p = \frac{n + 1}{100} \cdot p$$

2.3.0.4 quantile()

```
quantile(x, probs = c(0, .25, .50, .75, 1))
```

Funzione per il calcolo dei quantili di diverso ordine (definito dall'argomento probs)

```
set.seed(123) # garantisce la replicabilità
x = rnorm(1000, mean = 0, sd = 1)
quantile(x)
```

```
0%          25%          50%          75%          100%
-2.809774679 -0.628324243  0.009209639  0.664601867  3.241039935
```

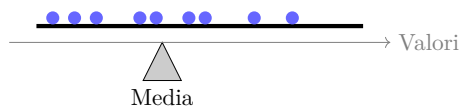
```
# calcola i decili
quantile(x, probs = seq(0,1, by = .10))
```

0%	10%	20%	30%	40%	50%
-2.809774679	-1.267328289	-0.788690220	-0.486452454	-0.214854912	0.009209639
60%	70%	80%	90%	100%	
0.242040647	0.513088142	0.841412710	1.254751947	3.241039935	

2.3.0.5 Media

La media è una sorta di baricentro della distribuzione dei dati. Data una variabile X composta da n valori:

$$\bar{X} = \frac{\sum_{i=1}^n x_i}{n}$$



I dati: $X = (0.8, 1.2, 1.6, 2.4, 2.7, 3.3, 3.6, 4.5, 5.2)$, la media $\bar{X} = 2.811$



I dati: $X = (1.18, 1.02, 1.10, 1.16, 1.01, 1.10, 1.05, 1.12, 1.45, 1.52, 1.54, 1.55, 10.06, 14.46)$, la media $\bar{X} = 2.809$

2.3.0.6 mean()

```
mean(x, na.rm = FALSE)
```

Funzione per calcolare la media di un vettore di valori x . L'argomento `na.rm = TRUE` serve per ottenere il calcolo della media nonostante la presenza di valori NA

`rowMeans()`: Calcola la media per ogni riga (quindi attraverso le colonne)

`colMeans()`: Calcola la media per ogni colonna (quindi attraverso le righe)

```
mean(x)
```

```
[1] 0.01612787
```

Table 2.1: Dataset di Anscombe

Dataset	Media.X	Media.Y	Varianza.X	Varianza.Y	Correlazione
Dataset 1	9	7.5	11	4.13	0.82
Dataset 2	9	7.5	11	4.13	0.82
Dataset 3	9	7.5	11	4.12	0.82
Dataset 4	9	7.5	11	4.12	0.82

2.4 Perché i grafici

Graphs remind us that the process of statistical inference is not mechanical. This process often involves subjective decisions (e.g., the evaluation, exploration, and/or deletion of outliers) which are an integral part of the analysis. Thus, graphs are among the most appropriate tools to enhance transparency and confer plausibility to such decisions.

Pastore et al. (2017)

Data set di Anscombe: Quattro dataset, di seguito le statistiche descrittive relative alla variabile x e alla variabile y e la loro correlazione:

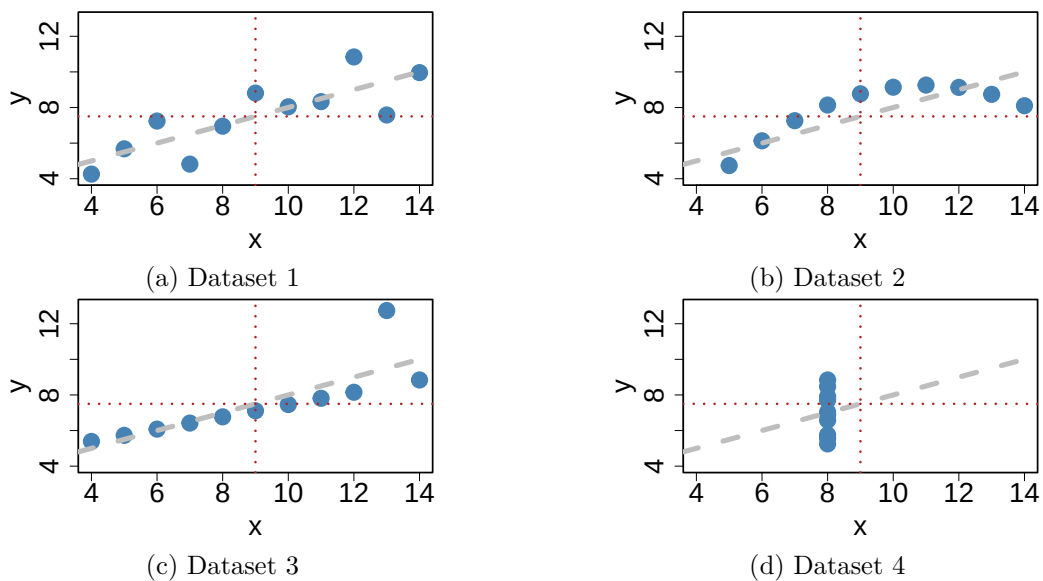


Figure 2.1: I grafici raccontano un'altra storia

2.5 Grafici

R fornisce uno strumento molto utile per la rappresentazione grafica dei dati e dei risultati già con il suo pacchetto base `graphics`

NB

Non avete bisogno di installarlo, è caricato automaticamente nel momento in cui scaricate R e le sue funzioni sono sempre disponibili

`ggplot2` è un pacchetto più avanzato per la creazione dei grafici. Non entreremo nel dettaglio del suo funzionamento perché è un po' più complicato

Alcune funzioni incluse nel pacchetto `graphics`:

<code>plot(x)</code>	①
<code>plot(x, y)</code>	②
<code>hist(x)</code>	③
<code>boxplot(x)</code>	④
<code>barplot(x)</code>	⑤

- ① Grafico di una variabile quantitativa
- ② Grafico a dispersione di due variabili quantitative
- ③ Istogramma di una variabile quantitativa
- ④ Boxplot della distribuzione quartile di una (o più) variabili
- ⑤ Grafico a barre per le frequenze delle variabili qualitative

2.5.1 Grafico a barre

Usato per variabili qualitative nominali o ordinali:

- Asse x : categoria k
- Asse y : Frequenza n_k (vale anche per le frequenze percentuali)

L'altezza della barretta è proporzionale alla sua frequenza

`barplot(x)`

Dove x è il vettore delle frequenze (quello che si ottiene applicando la funzione `table()` a una variabile categoriale)

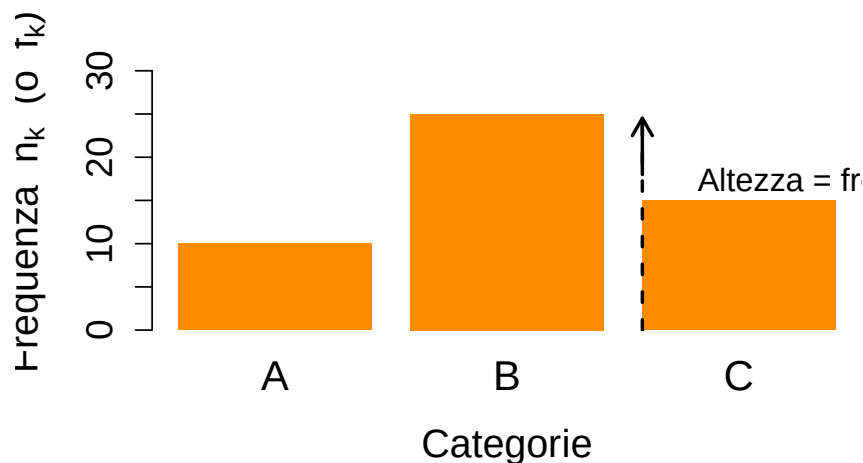


Table 2.2: Tabelle di frequenza per le categorie A, B, C.

Categoria	n_k
A	21
B	24
C	22

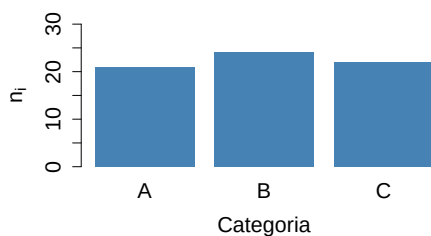


Figure 2.2: Barplot delle tabelle

Categoria	n_k
A	12
B	35
C	18

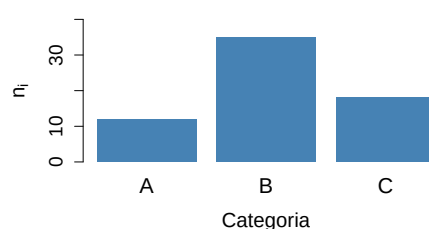


Figure 2.3: Barplot delle tabelle

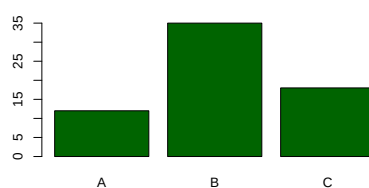
```
dati = factor(rep(c("A", "B", "C"), c(12, 35, 18))); dati
```

```
[1] A A A A A A A A A A A A B B B B B B B B B B B B B B B B B B B B B B
[39] B B B B B B B B B C C C C C C C C C C C C C C C C C C C C
Levels: A B C
```

```
table(dati)
```

```
dati
 A  B  C
12 35 18
```

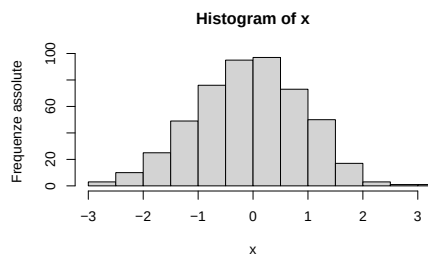
```
barplot(table(dati), col = "darkgreen")
```



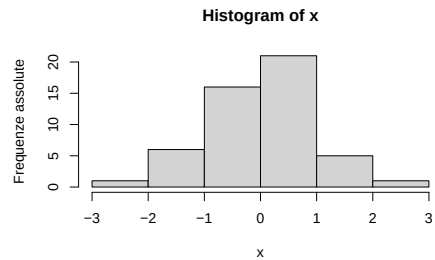
2.5.2 Istogrammi

`hist(x, breaks)` Produce un istogramma a partire da un vettore `x` di osservazioni quantitative. `breaks` definisce la larghezza delle singole barre, secondo diverse regole

```
x = rnorm(500, mean = 0, sd = 1)
hist(x)
```

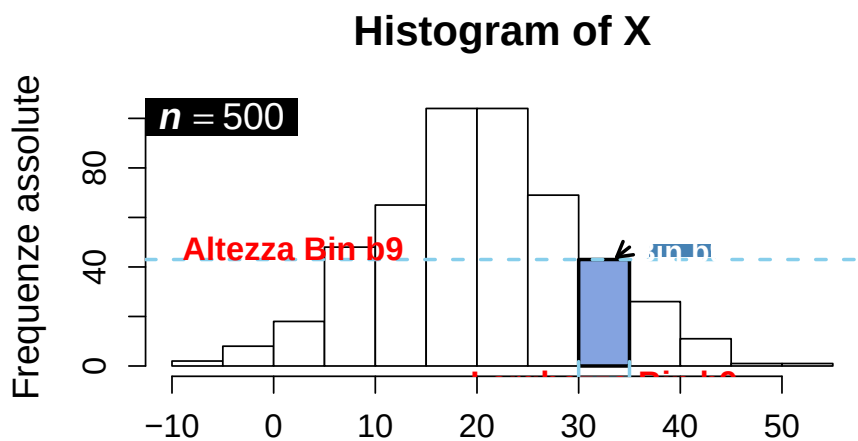
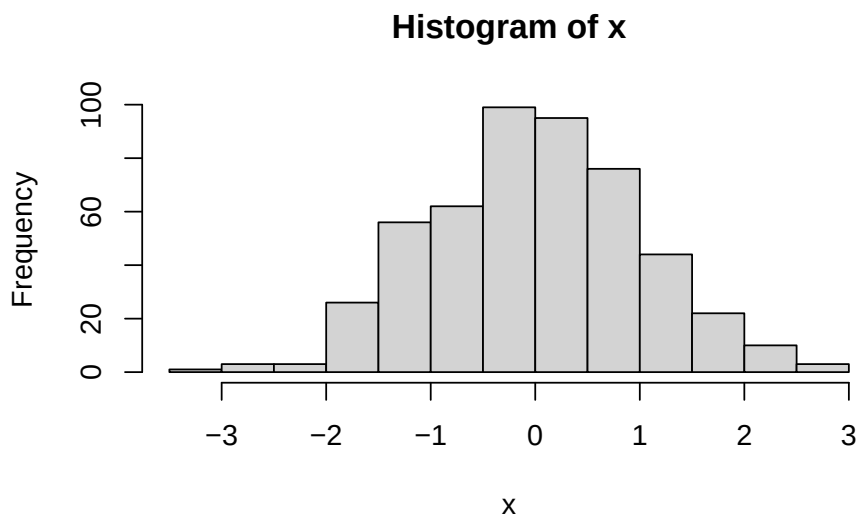


(a) $n = 500$



(a) $n = 50$

Istogrammi con n diverso



Sturges:

$$\frac{\max(x) - \min(x)}{\log_2(n + 1)}$$

Scott:

$$3.5 \sqrt{\text{var}(x)} n^{-1/3}$$

Freedman-Diaconis

$$2 (Q_3 - Q_1) n^{-1/3}$$

User defined:

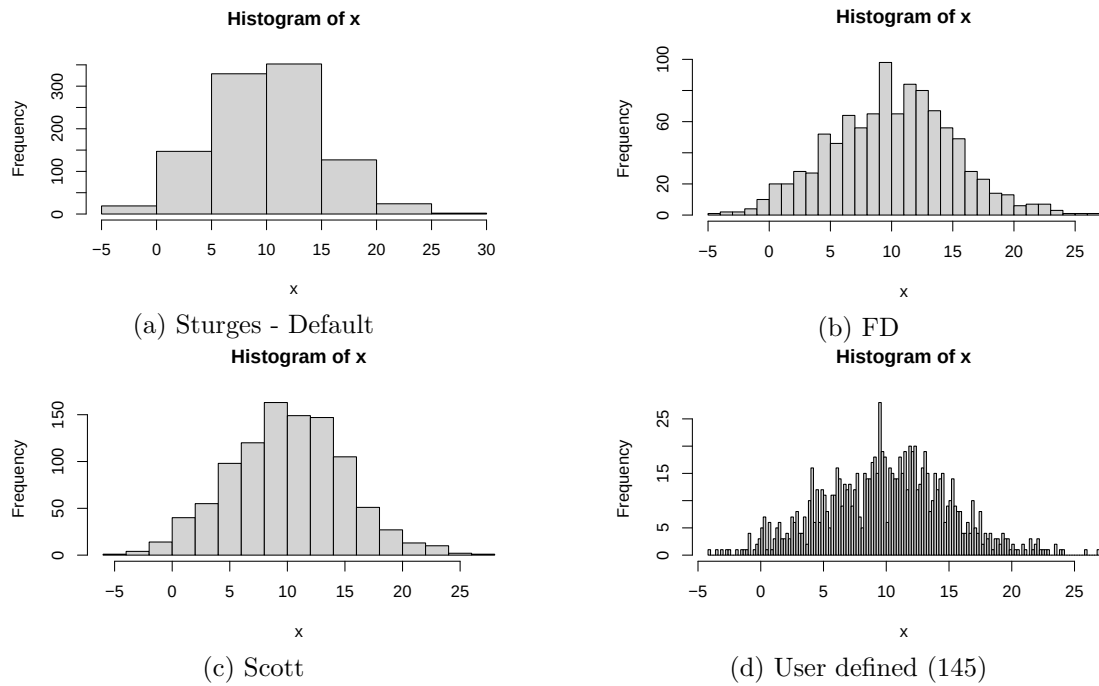
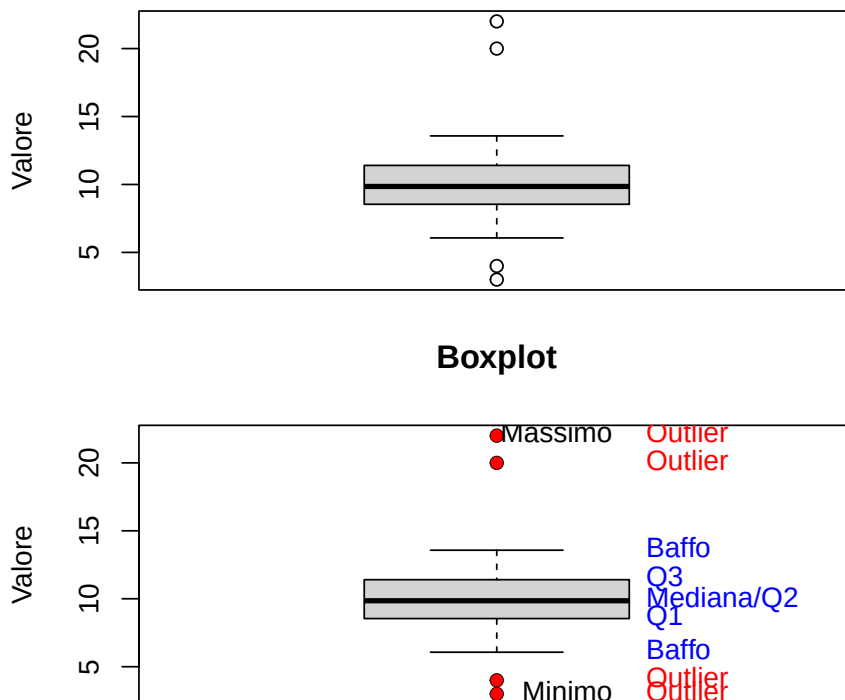


Figure 2.6: Metodi per la definizione dei breaks

$$\frac{\max(x) - \min(x)}{\text{breaks}}$$

2.5.3 Boxplot

`boxplot(x)` Produce un boxplot a partire da un vettore X di osservazioni quantitative



2.5.4 Boxplot – Calcolo degli outlier

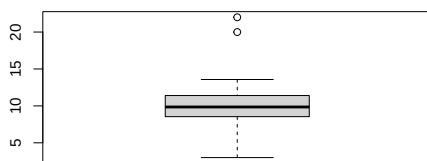
La lunghezza dei baffi (e di conseguenza la definizione degli outliers) è definita attraverso il metodo Tukey:

$$[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$$

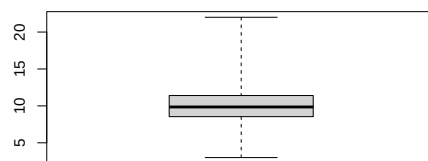
dove $IQR = Q_3 - Q_1$ (Inter-quartile range)

Settando l'argomento `range = x` si può cambiare il modo in cui vengono calcolati i baffi:

```
boxplot(x, range = 2)
boxplot(x, range = 0)
```



(a) range = 2 - Meno Conservativo
Calcolo degli outlier



(a) range = 0 - Minimo e Massimo

2.6 Esempi

2.6.0.1 barplot()

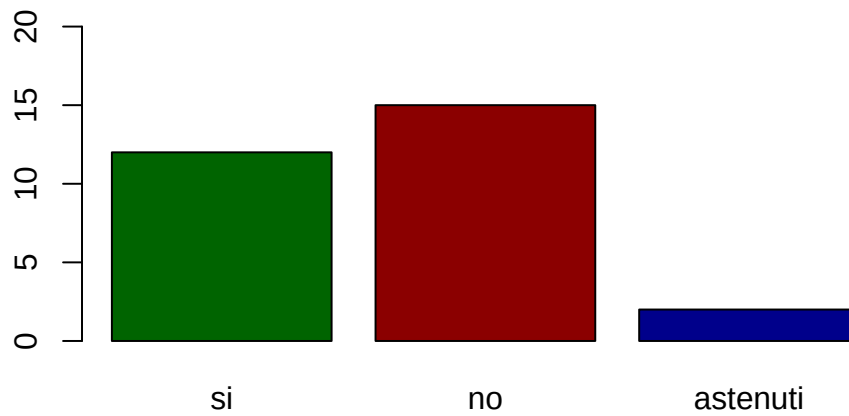
```
voto = sample(rep(c("si", "no", "astenuti"),
                  c(12, 15, 2)))
voto = factor(voto, levels = c("si", "no", "astenuti"))
table(voto)

# le frequenze restituite dalla funzione table() vanno usate
# come vettore di base per il barplot

barplot(table(voto),
        col = c("darkgreen", "darkred", "darkblue"),
        ylim = c(0, 20))
```

```
voto
      si      no astenuti
      12      15         2
```

```
barplot(table(voto),
        col = c("darkgreen", "darkred", "darkblue"),
        ylim = c(0, 20))
```



2.6.0.2 boxplot()

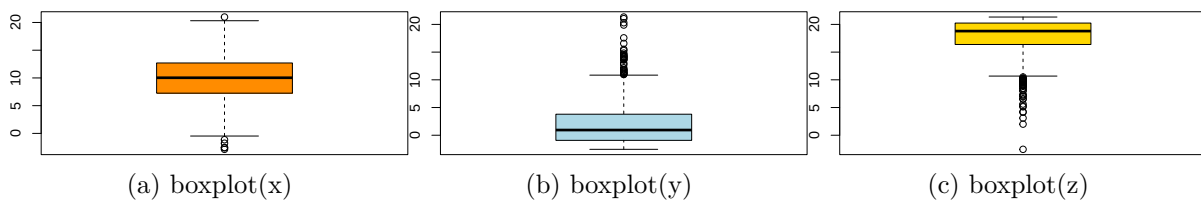


Figure 2.9: Distribuzioni a confronto

Figure 2.9a: La distribuzione è equilibrata attorno alla mediana, situata circa a metà della distribuzione tra il minimo e il massimo

Figure 2.9b: La distribuzione è squilibrata verso il basso, ovvero verso il minimo, presentando una coda più “pesante” verso il massimo

Figure 2.9c: La distribuzione è squilibrata verso l’alto, ovvero verso il massimo, presentando una coda più “pesante” verso il minimo

2.6.0.3 hist()

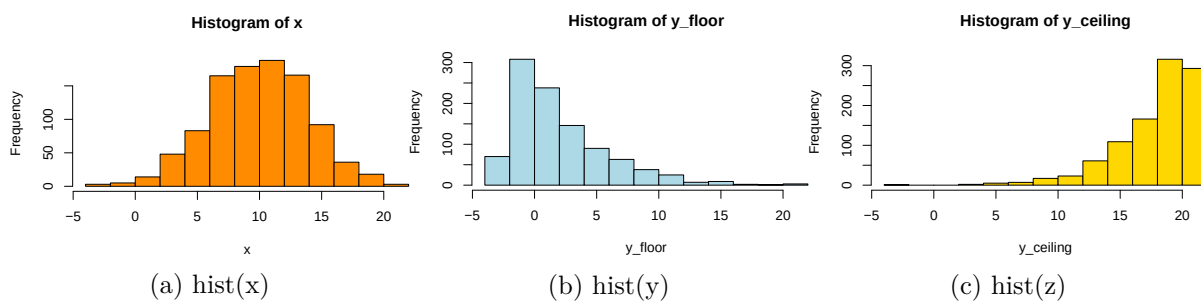


Figure 2.10: Distribuzioni a confronto - Le stesse dei boxplot

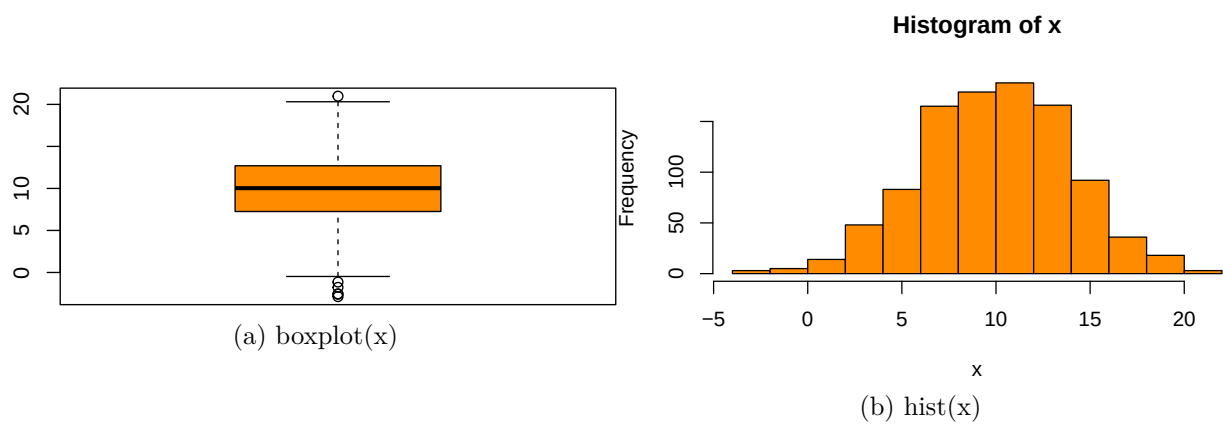


Figure 2.11: Boxplot vs Istogramma

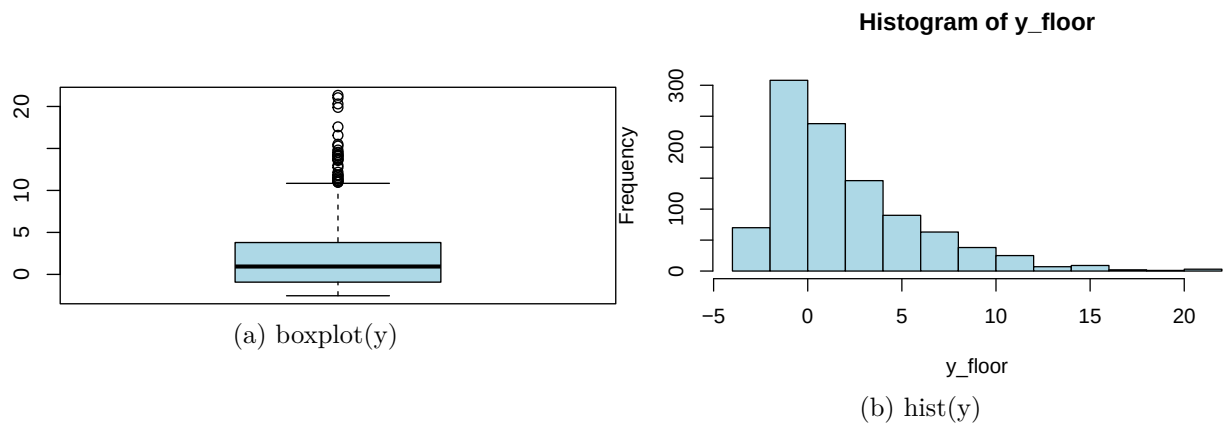


Figure 2.12: Boxplot vs Istogramma

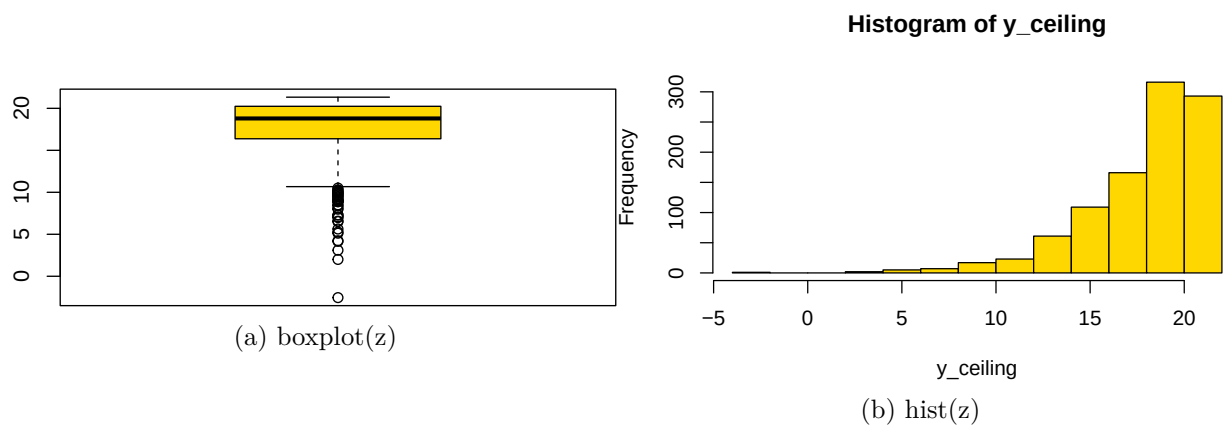
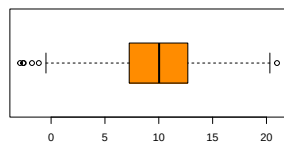
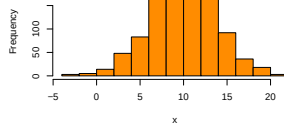


Figure 2.13: Boxplot vs Istogramma

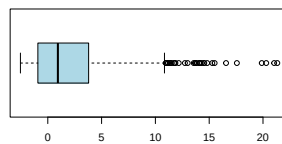


(a)

Histogram of x

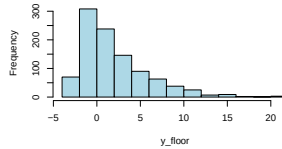


(d)

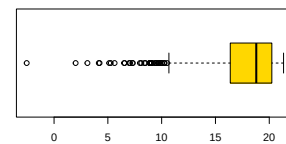


(b)

Histogram of y_floor

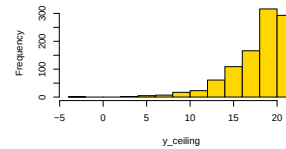


(e)



(c)

Histogram of y_ceiling



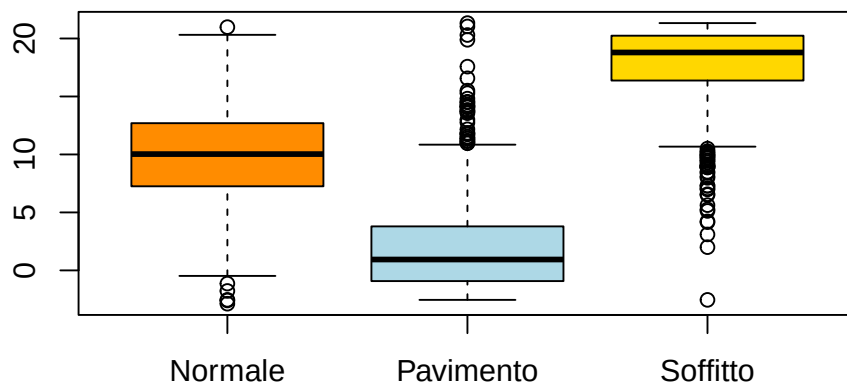
(f)

2.6.0.4 Confronto boxplot vs istogramma

2.6.0.5 Confronto boxplot vs istogramma

2.6.0.6 Confronto boxplot vs istogramma

```
boxplot(x, y, z, names = c("Normale", "Pavimento", "Soffitto"),
        col = c("darkorange", "lightblue", "gold"))
```



3 Modulo 3 – Statistica descrittiva bivariata

3.1 Variabile statistica doppia

X: Istruzione	E	E	E	SI	SI	SI	SS	SS	SS	SS	SS
Y: Occupazione	DI	I	I	DI	I	I	D	I	I	D	D

categoria x_k	$n_{k.}$
Elementare (E)	3
Secondaria Inferiore (SI)	3
Secondaria Superiore (SS)	5

categoria y_j	$n_{.j}$
Disoccupato (DI)	2
Impiegato (I)	6
Dirigente (D)	3

	DI	I	D	
E	1	2	0	3
SI	1	2	0	3
SS	0	2	3	5
	2	6	3	11

Date due variabili X e Y e la loro distribuzione di frequenze congiunte n_{kj} (numero di unità statistiche che presentano la coppia di categorie (x_k, y_j))

Tip

L'insieme (X, Y) delle terne

$$(x_k, y_j, n_{kj}) \quad (k = 1, \dots, h; j = 1, \dots, m)$$

è detto **variabile statistica doppia**.

3.2 Tabella a doppia entrata – Tavola di contingenza

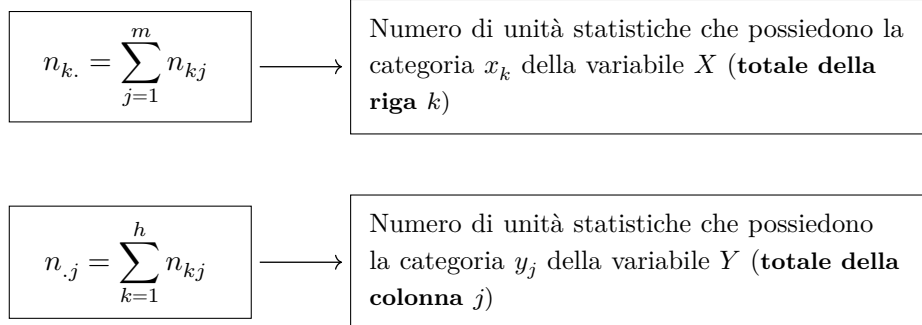
		Y					
		y_1	y_2	...	y_j	...	y_m
X	x_1	n_{11}	n_{12}	...	n_{1j}	...	n_{1m}
	x_2	n_{21}	n_{22}	...	n_{2j}	...	n_{2m}
	$\vdots$	$\vdots$	$\vdots$		$\vdots$		$\vdots$
	x_k	n_{k1}	n_{k2}	...	n_{kj}	...	n_{km}
	$\vdots$	$\vdots$	$\vdots$		$\vdots$		$\vdots$
	x_h	n_{h1}	n_{h2}	...	n_{hj}	...	n_{hm}
		$n_{.1}$	$n_{.2}$	...	$n_{.j}$	...	$n_{.m}$

n_{kj} : **Frequenza congiunta**, definisce il numero di unità statistiche che possiedono congiuntamente la categoria x_k della variabile X e la categoria y_j della variabile Y

$n_{k.}$: **Frequenza marginale di X** , definisce il numero di unità statistiche che possiedono la categoria x_k della variabile X

$n_{.j}$: **Frequenza marginale di Y** , definisce il numero di unità statistiche che possiedono la categoria y_j della variabile Y

n : Numero totale di unità statistiche



$$\sum_{k=1}^h \sum_{j=1}^m n_{kj} = \sum_{k=1}^h n_{k.} = \sum_{j=1}^m n_{.j} = n$$

3.2.1 table()

table(var1, var2): Produce la tabella di frequenza tra due variabili categoriali var1 e var2

```
var1 <- as.factor(c("a1","a2","a1","a2","a1","a2","a1"))
var2 <- as.factor(c("b1","b1","b1","b2","b1","b2","b2"))
mytab = table(var1, var2); mytab
```

```
      var2
var1 b1 b2
a1   3  1
a2   1  2
```

```
sum(mytab) # permette di ottenere l'ampiezza campionaria
```

```
[1] 7
```

3.2.2 margin.table()

margin.table(table, margin): Frequenze marginali per la tabella a doppia entrata

Tip

margin = 1: Frequenze marginali calcolate **attraverso le colonne**
margin = 2: Frequenze marginali calcolate **attraverso le righe**

```
mytab
```

```
      var2
var1 b1 b2
a1   3  1
a2   1  2
```

```
margin.table(mytab,1)
```

```
var1
a1 a2
4  3
```

```
margin.table(mytab,2)
```

```
var2
b1 b2
4  3
```

3.2.3 prop.table()

prop.table(T, margin = NULL): Probabilità condizionate:

Tip

margin = 1: Probabilità condizionate di riga (**attraverso le colonne**)

margin = 2: Probabilità condizionate di colonna (**attraverso le righe**)

```
mytab
```

```
      var2
var1 b1 b2
a1   3  1
a2   1  2
```

```
prop.table(mytab)
```

```
      var2
var1      b1      b2
a1 0.4285714 0.1428571
a2 0.1428571 0.2857143
```

```
prop.table(mytab,1) # P(B/A)
```

```

      var2
var1      b1      b2
a1 0.7500000 0.2500000
a2 0.3333333 0.6666667

```

```
prop.table(mytab,2) #  $P(A/B)$ 
```

```

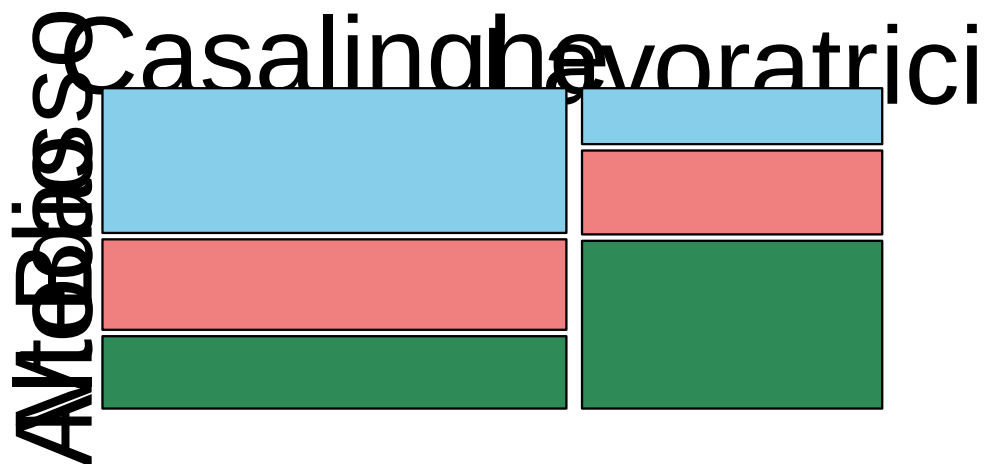
      var2
var1      b1      b2
a1 0.7500000 0.3333333
a2 0.2500000 0.6666667

```

3.2.4 Frequenze percentuali

	Basso	Medio	Alto	Totale
Casalinghe	40	25	20	85
Lavoratrici	10	15	30	55
Totale	50	40	50	140

tab



3.2.5 $P(\text{Titolo}|\text{Occupazione})$ (Frequenze percentuali di riga)

	Basso	Medio	Alto	Sum
Casalinghe	47	29	24	100
Lavoratrici	18	27	55	100

Frequenze assolute per Occupazione e Livello di Istruzioni

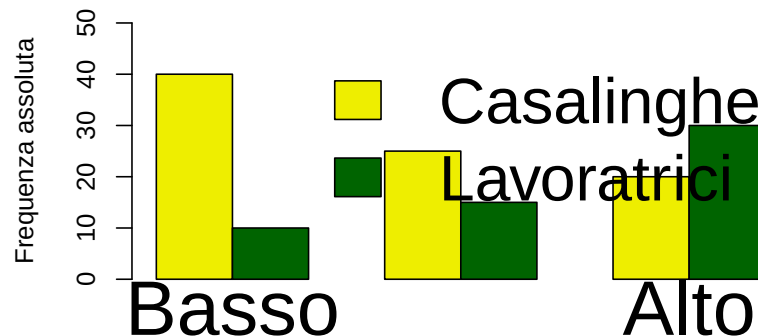


X% delle donne lavoratrici/casalinghe hanno un titolo di studio basso/medio/alto

3.2.6 $P(\text{Occupazione}|\text{Titolo})$ (Frequenze percentuali di colonna)

	Basso	Medio	Alto
Casalinghe	80	62	40
Lavoratrici	20	38	60
Sum	100	100	100

Frequenze assolute per Occupazione e Livello di Istruzioni



X% delle donne con titolo di studio basso/medio/alto che sono lavoratrici/casalinghe

3.3 Connessione (Variabili qualitative)

La CONNESSIONE tra due variabili categoriali equivale alla NON INDIPENDENZA!

Tra le variabili X e Y c'è una forma di legame e **relazione (connessione)** → il variare di una implica un qualche cambiamento **non casuale** nell'altra

3.3.1 χ^2

Quantifica la DISTANZA DALL'INDIPENDENZA tra due variabili categoriali

- X con h categorie x_k , con $k = 1, \dots, h$,
- Y con m categorie y_j , con $j = 1, \dots, m$

Richiede il confronto tra **frequenze osservate** (n_{kj}) e **frequenze attese** o **frequenze teoriche** (n_{kj}^*)

n_{kj}	$n_{k.}$
$n_{.j}$	

Frequenze osservate

VS.

n_{kj}^*	$n_{k.}$
$n_{.j}$	

Frequenze teoriche

$$n_{kj}^* = \frac{n_{k.} \times n_{.j}}{n}, \quad n = \sum_{k=1}^h \sum_{j=1}^m n_{kj} = \sum_{k=1}^h \sum_{j=1}^m n_{kj}^*$$

n_{kj}^* : Significato

Data l'ampiezza campionaria n , quali sono le frequenze che avremmo dovuto osservare se le due variabili fossero state indipendenti, ovvero se non avessero alcuna relazione?

$$n_{kj}^* = \frac{n_{k.} \cdot n_{.j}}{n}$$

```
knitr::kable(tabella,booktabs = TRUE,align = "c")
```

Table 3.4: Frequenze osservate

	Basso	Medio	Alto	Totale
Casalinghe	40	25	20	85
Lavoratrici	10	15	30	55
Totale	50	40	50	140

$$\chi^2 = \sum_{k=1}^h \sum_{j=1}^m \frac{(n_{kj} - n_{kj}^*)^2}{n_{kj}^*}$$

Table 3.5: Frequenze teoriche

	Basso	Medio	Alto	Totale
Casalinghe	$\frac{85 \times 50}{140} = 30.36$	$\frac{85 \times 40}{140} = 24.29$	$\frac{85 \times 50}{140} = 30.36$	85
Lavoratrici	$\frac{55 \times 50}{140} = 19.64$	$\frac{55 \times 40}{140} = 15.71$	$\frac{55 \times 50}{140} = 19.64$	55
Totale	50	40	50	140

$\chi^2 = 0$: $n_{kj} = n_{kj}^*$ Indipendenza stocastica

$\chi^2 = \chi_{\max}^2$: Relazione di perfetta dipendenza (dove $\chi_{\max}^2 = n \times \min[(h-1); (m-1)]$)

$$\tilde{\chi}^2 = \frac{\chi^2}{\chi_{\max}^2}$$

$\tilde{\chi}^2$ è il χ^2 normalizzato, permette di esprimere il grado di associazione tra due variabili categoriali secondo una scala di tipo $[0, 1]$

3.3.2 chisq.test()

`chisq.test(table)$statistic`: Calcola il χ^2 osservato a partire da una tavola di contingenza ottenuta con `table()`

```
mytab
```

```
      var2
var1 b1 b2
a1   3  1
a2   1  2
```

```
chisq.test(mytab)$statistic
```

```
X-squared
0.109375
```

3.3.2.1 Calcolo $\chi_{\max}^2$

```
chiMax = sum(mytab) * min(c(nrow(mytab) - 1, ncol(mytab) - 1))
chiMax
```

```
[1] 7
```

```
chisq.test(mytab)$statistic/chiMax
```

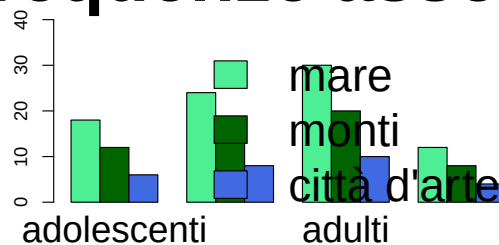
Warning in `chisq.test(mytab)`: L'approssimazione al Chi-quadrato potrebbe essere inesatta

X-squared
0.015625

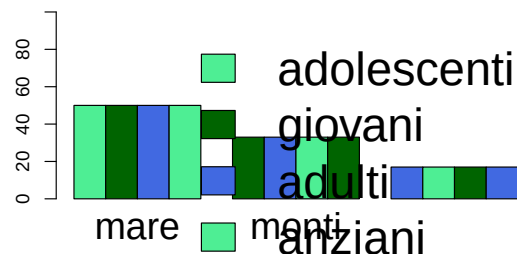
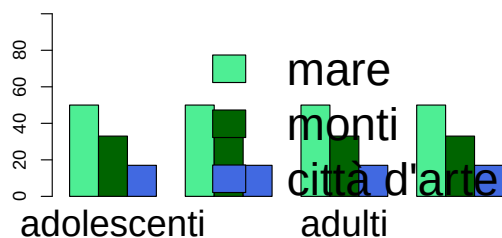
3.4 Indipendenza stocastica e massima dipendenza

	adolescenti	giovani	adulti	anziani
mare	18	24	30	12
monti	12	16	20	8
città d'arte	6	8	10	4

Frequenze assolute



Percentuali di colorePercentuali di riga



Frequenze osservate

```
obs_n = matrix(
  c(18, 24, 30, 12, # mare
    12, 16, 20, 8,   # monti
    6, 8, 10, 4),    # città d'arte
  nrow = 3, byrow = TRUE) ; obs_n
```

```
      [,1] [,2] [,3] [,4]
[1,]   18  24  30  12
[2,]   12  16  20   8
[3,]    6   8  10   4
```

Frequenze teoriche

```
chisq.test(obs_n)$expected
```

```
      [,1] [,2] [,3] [,4]  
[1,]    18    24    30    12  
[2,]    12    16    20     8  
[3,]     6     8    10     4
```

3.4.0.1 Calcolo χ^2

```
chisq.test(obs_n)$statistic
```

Warning in chisq.test(obs_n): L'approssimazione al Chi-quadrato potrebbe essere inesatta

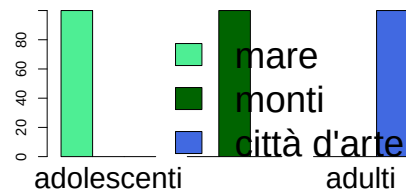
```
X-squared  
      0
```

```
sum(obs_n) * min(c(nrow(obs_n) - 1,  
                    ncol(obs_n) - 1))
```

```
[1] 336
```

	adolescenti	giovani	adulti	Sum
mare	84	0	0	84
monti	0	64	0	64
città d'arte	0	0	20	20
Sum	84	64	20	168

Percentuali di color



Frequenze osservate

```
obs_n <- matrix(  
  c(84, 0, 0,      # mare → solo adolescenti  
    0, 64, 0,      # monti → solo giovani  
    0, 0, 20),     # città d'arte → solo adulti  
  nrow = 3, byrow = TRUE) ; obs_n
```

```

      [,1] [,2] [,3]
[1,]   84    0    0
[2,]    0   64    0
[3,]    0    0   20

```

Frequenze teoriche

```
chisq.test(obs_n)$expected
```

```

      [,1]      [,2]      [,3]
[1,]   42 32.000000 10.000000
[2,]   32 24.380952  7.619048
[3,]   10  7.619048  2.380952

```

Table 3.8: Frequenze osservate

84	0	0
0	64	0
0	0	20

Table 3.9: Frequenze Teoriche

42	32.000000	10.000000
32	24.380952	7.619048
10	7.619048	2.380952

```
chisq.test(obs_n)$statistic
```

Warning in `chisq.test(obs_n)`: L'approssimazione al Chi-quadrato potrebbe essere inesatta

```

X-squared
      336

```

```

sum(obs_n) * min(c(nrow(obs_n) - 1,
                    ncol(obs_n) - 1))

```

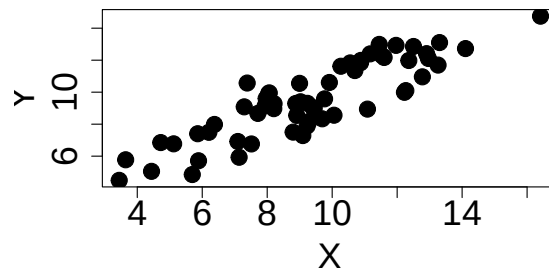
```
[1] 336
```

3.5 Associazione (variabili quantitative)

3.5.1 Relazione lineare

Si vuole studiare la **relazione lineare** tra due variabili quantitative X e Y misurate sulle stesse unità statistiche

Per relazione lineare si intende quanto le coppie di punti (x_i, y_i) tendono a disporsi lungo una retta



Due indici: la **covarianza** (direzione della relazione) e il **coefficiente di correlazione lineare di Bravais-Pearson** (direzione e intensità)

3.5.2 Indice di covarianza campionaria

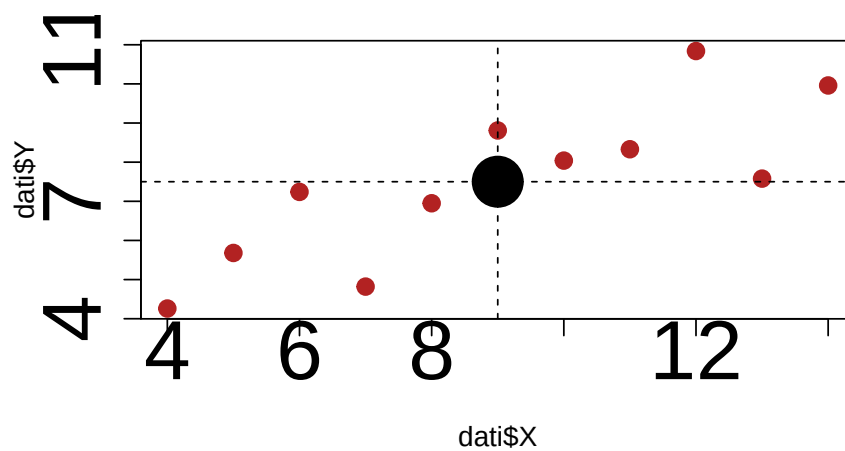
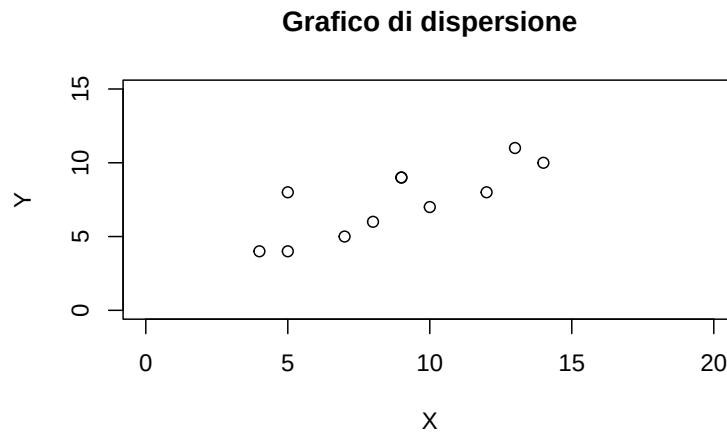


Figure 3.1: Grafico a dispersione delle variabili X e Y

	X	Y
1	10	8.04
2	8	6.95
3	13	7.58
4	9	8.81
5	11	8.33
6	14	9.96
7	6	7.24
8	4	4.26
9	12	10.84
10	7	4.82
11	5	5.68

`plot(x, y)`: Grafico di dispersione di due variabili quantitative appaiate (x,y) aventi lo stesso numero di osservazioni

```
X <- c(9,8,12,9,10,14,5,4,13,7,5)
Y <- c(9,6,8,9,7,10,8,4,11,5,4)
plot(X,Y,main="Grafico di dispersione", xlim=c(0,20),ylim=c(0,15))
```



La **Covarianza** misura il grado di *dispersione* e *covariazione* di due variabili quantitative appa-
iate (X, Y) attorno ai loro baricentri (medie) $\bar{X}$ e $\bar{Y}$

$$x = (x_1, x_2, \dots, x_n)$$

$$y = (y_1, y_2, \dots, y_n)$$

$$d = ((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$$

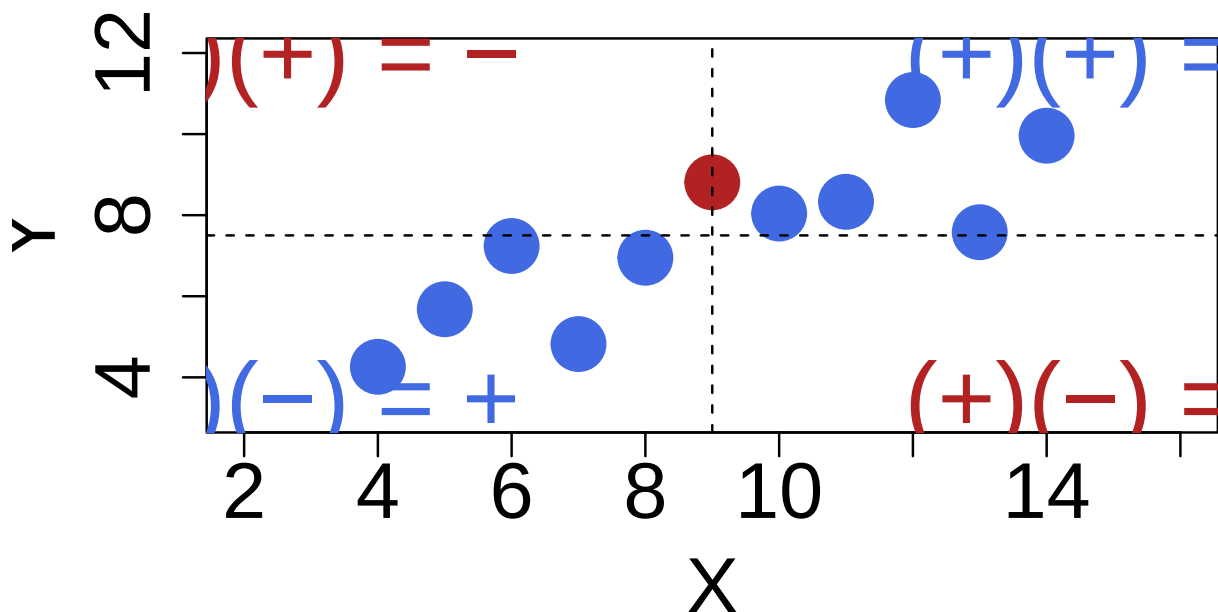
$$\text{Cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

È (quasi) la **media dei prodotti degli scarti** di ogni variabile dalla propria media

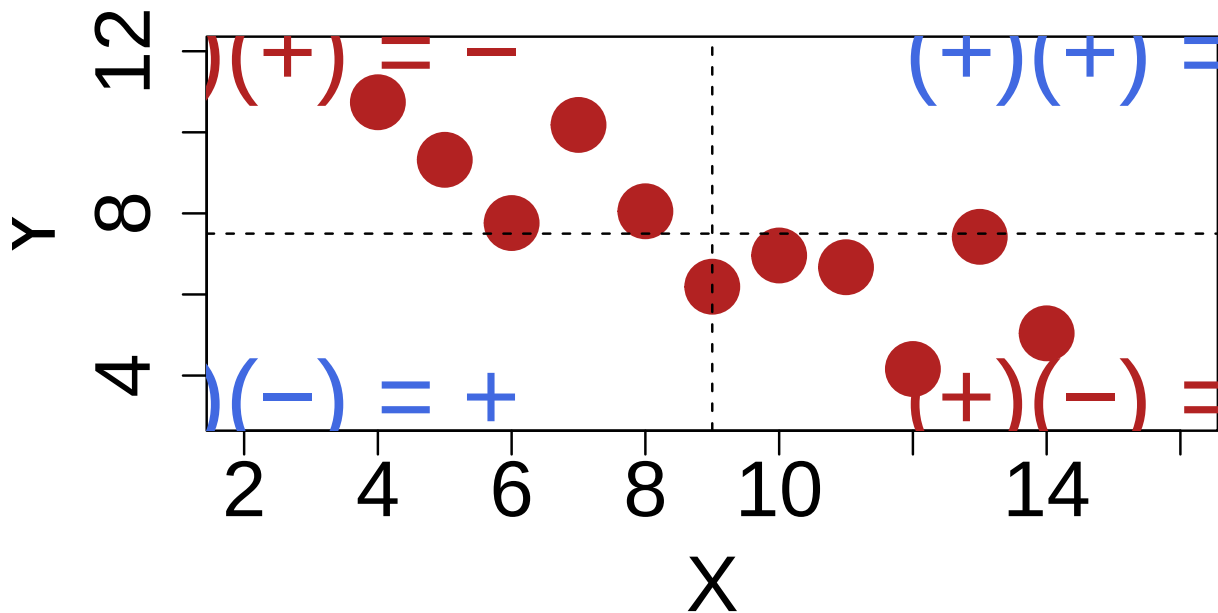
Note

Nota: alcuni testi dividono per n invece che per $n - 1$ (covarianza “descrittiva”). La funzione `cov()` di R divide per $n - 1$, quindi useremo sempre questa versione

$$\text{Cov}(X, Y) > 0$$



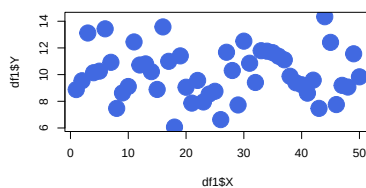
$$\text{Cov}(X, Y) < 0$$



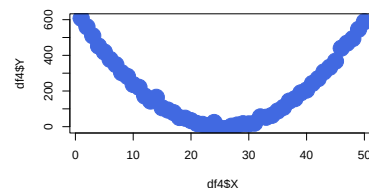
Il prodotto $(x_i - \bar{x})(y_i - \bar{y})$ è più spesso positivo se $\text{Cov} > 0$, più spesso negativo se $\text{Cov} < 0$

- La covarianza può assumere **tutti i valori reali**
- $\text{Cov}(X, Y) > 0$: tendenza alla linearità positiva (**crescente**)
- $\text{Cov}(X, Y) < 0$: tendenza alla linearità negativa (**decrecente**)
- $\text{Cov}(X, Y) \approx 0$: nessuna tendenza lineare

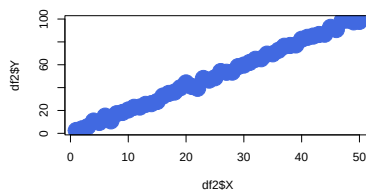
Il valore della covarianza dipende dall'unità di misura delle variabili → ci dice la **direzione** della relazione ma non la sua **intensità**



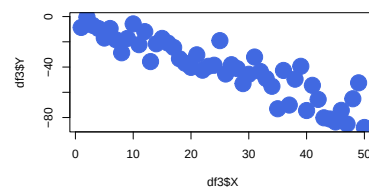
(a) $\text{Cov} = 0$



(a) $\text{Cov} = 0$



(a) $\text{Cov} > 0$



(a) $\text{Cov} < 0$

Covarianza

Età (in mesi) e peso (in kg) di un bambino rilevati in 6 momenti

	eta	peso
1	3	6
2	6	8
3	9	9
4	12	10
5	24	13
6	36	20

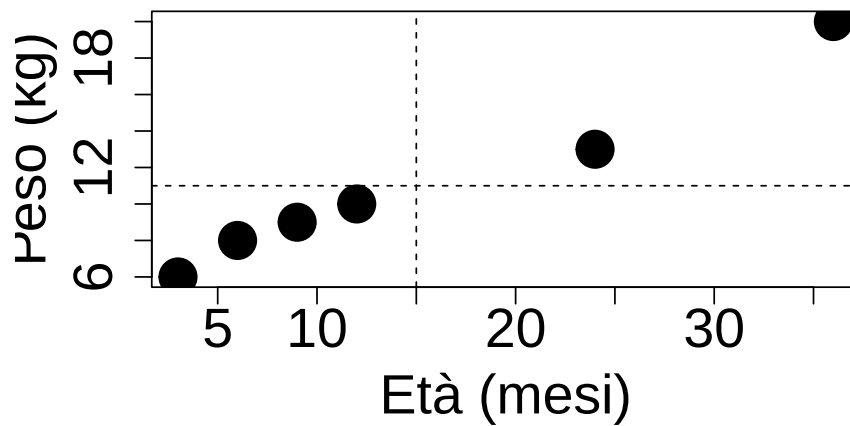


Figure 3.6: Grafico a dispersione

3.5.2.1 Calcolo della covarianza

	x	y	x - media(x)	y - media(y)	prodotto
1	3	6	-12	-5	60
2	6	8	-9	-3	27
3	9	9	-6	-2	12
4	12	10	-3	-1	3
5	24	13	9	2	18
6	36	20	21	9	189

$$\bar{x} = 15, \bar{y} = 11, \sum_i (x_i - \bar{x})(y_i - \bar{y}) = 309$$

$$\text{Cov}(x, y) = \frac{309}{6-1} = 61.8$$

E quindi? Sappiamo che la relazione è **positiva**, ma è forte o debole?

3.5.3 cov()

cov(): Restituisce la covarianza campionaria tra due variabili quantitative appaiate (X, Y) aventi lo stesso numero di osservazioni

```
X <- c(9,8,12,9,10,14,5,4,13,7,5)
Y <- c(9,6,8,9,7,10,8,4,11,5,4)
Z <- -Y
cov(X,Y)
```

```
[1] 6.309091
```

```
cov(X,Z)
```

```
[1] -6.309091
```

```
cov(bimbo$eta, bimbo$peso)
```

```
[1] 61.8
```

3.5.4 Correlazione campionaria

Per stabilire l'**intensità** della relazione la covarianza va *normalizzata*, ovvero divisa per il suo valore massimo

$$-sd(x)sd(y) \leq \text{Cov}(X, Y) \leq +sd(x)sd(y)$$

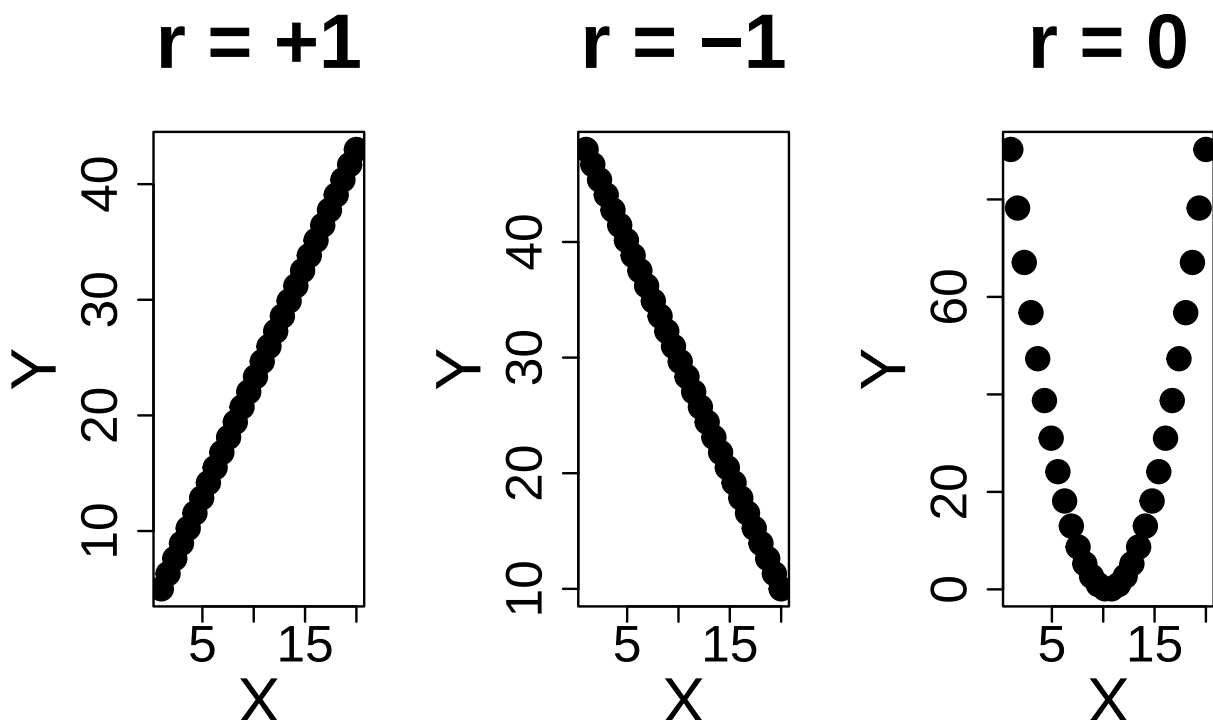
$$r = \frac{\text{Cov}(X, Y)}{sd(x)sd(y)}$$

$$-1 \leq r \leq 1$$

$r = +1$: legame **perfettamente lineare e diretto**;

$r = -1$: legame **perfettamente lineare e inverso**

$r = 0$: **incorrelazione**: non esiste un legame *lineare*... ma questo non implica che le variabili non siano in relazione tra loro



3.5.5 cor()

`cor()`: Restituisce la correlazione (lineare) campionaria tra due variabili quantitative appaiate (X, Y) aventi lo stesso numero di osservazioni

```
X <- c(9,8,12,9,10,14,5,4,13,7,5)
Y <- c(9,6,8,9,7,10,8,4,11,5,4)
Z <- -Y
cor(X,Y)
```

```
[1] 0.7921487
```

```
cor(X,Z)
```

```
[1] -0.7921487
```

3.5.5.1 Il bambino di prima

```
cov(bimbo$eta, bimbo$peso) / (sd(bimbo$eta) * sd(bimbo$peso))
```

```
[1] 0.9860185
```

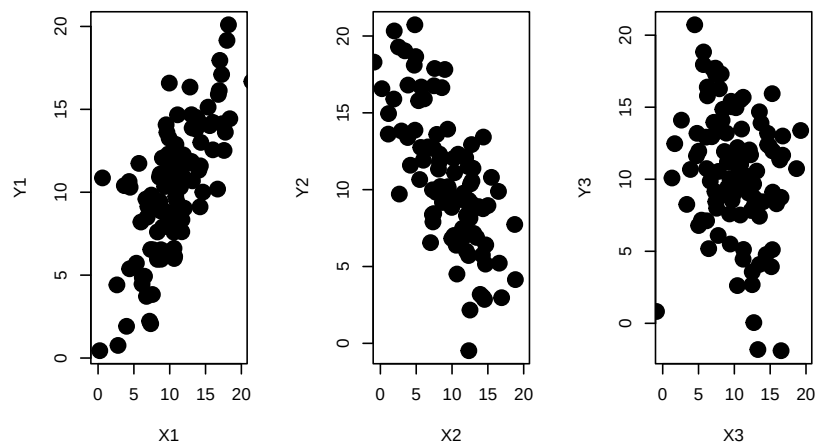
```
cor(bimbo$eta, bimbo$peso)
```

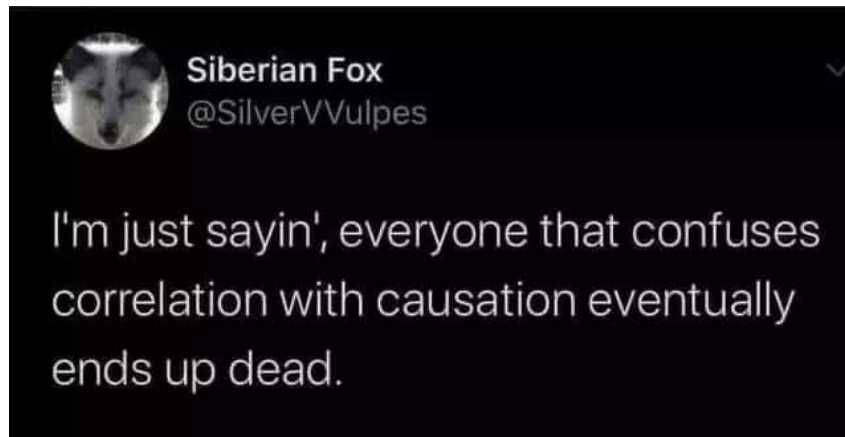
```
[1] 0.9860185
```

$$r = \frac{61.8}{12.59 \cdot 4.98} = 0.986$$

Le variabili presentano una **fortissima correlazione positiva**: i punti tendono a disporsi lungo una retta crescente

cov = 12.24 , r = 0.986 **cov = -13.09 , r = -0.986** **cov = -3.84 , r = -0.384**





3.5.6 Usi e abusi di r

3.5.6.1 In ψ

- Attendibilità test-retest
- Forme parallele di un test (coefficiente di equivalenza)
- Accordo tra le valutazioni di due giudici
- Consistenza interna di un questionario
- ...

3.5.6.2 Statistics doesn't lie, but liars use statistics

Due variabili possono essere fortemente correlate senza che esista alcun legame tra di loro

<https://www.tylervigen.com/spurious-correlations>

Correlazione $\neq$ causalità

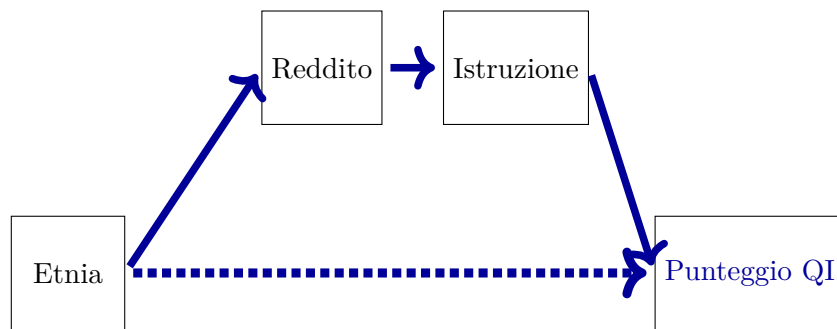
L'ignoranza afferma o nega rotondamente; la scienza dubita (Voltaire)

3.5.6.3 Relazione diretta



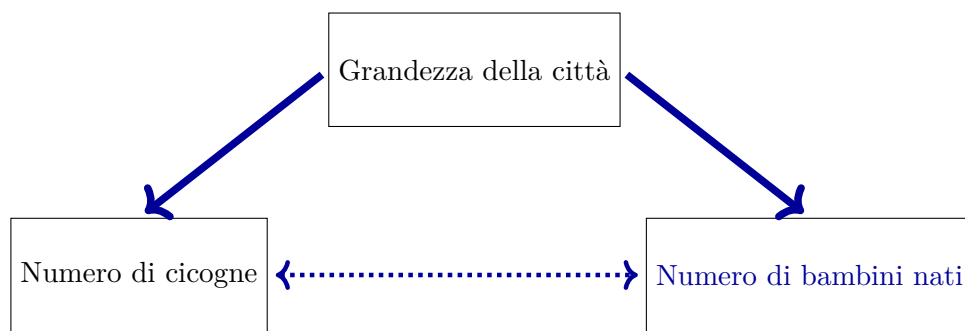
Ma ne siamo proprio sicuri?

3.5.6.4 Relazione indiretta



La relazione tra X e Y passa attraverso altre variabili (mediatori)

3.5.6.5 Relazione spuria



X e Y sono correlate solo perché entrambe dipendono da una terza variabile

LA STATISTICA SOSTIENE, NON ILLUMINA!

3.6 Previsione variabili quantitative

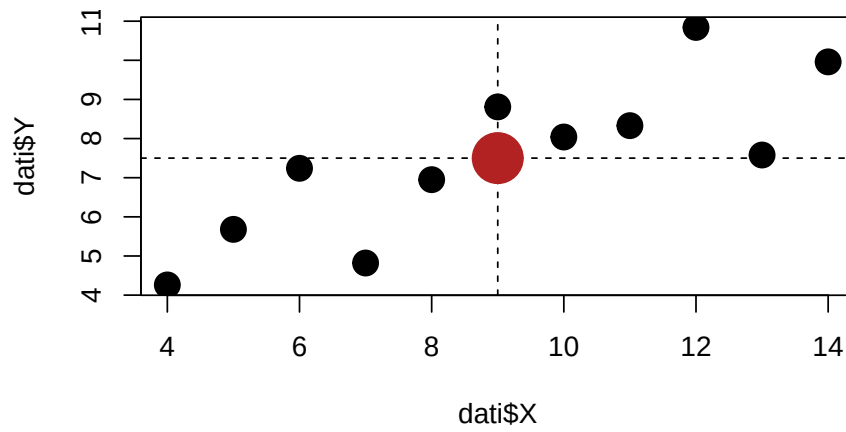
3.6.1 Regressione lineare

3.6.1.1 Quanto varia y in funzione di x ?

La correlazione permette di stabilire quanto covariano due variabili, x e y : **modello simmetrico** per determinare *l'intensità della relazione*

Con un modello di regressione si vuole andare più a fondo per capire **quanto** varia y in funzione di ogni cambiamento in x : stabilire non solo se le variabili sono associate o meno ma la portata della variazione

La relazione è *asimetrica*: si vuole spiegare y in funzione di x (questo non vuol dire in alcun modo che la relazione sia causale)



	X	Y
1	10	8.04
2	8	6.95
3	13	7.58
4	9	8.81
5	11	8.33
6	14	9.96
7	6	7.24
8	4	4.26
9	12	10.84
10	7	4.82
11	5	5.68

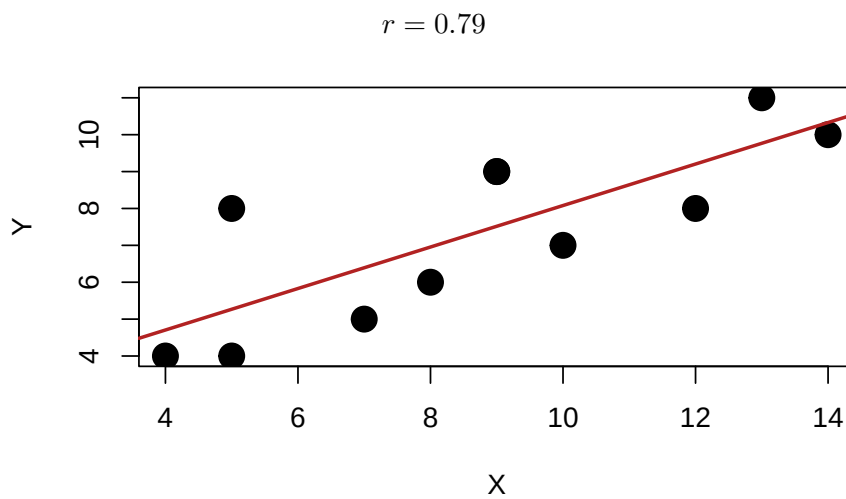


Figure 3.7: Una retta che passi attraverso le coppie di osservazioni

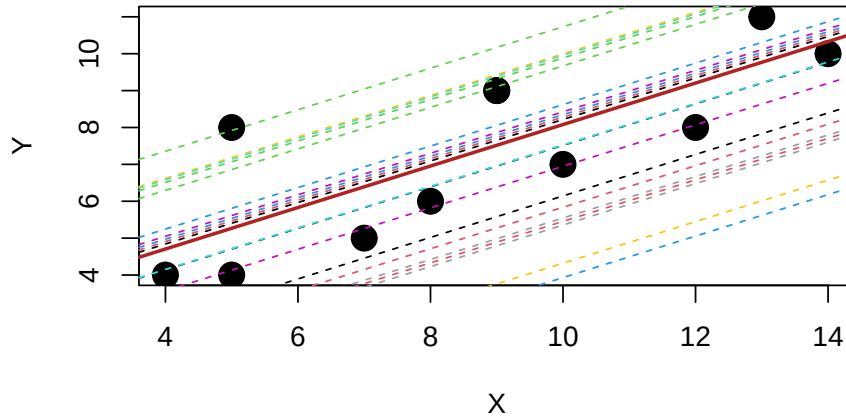


Figure 3.8: Ma quale retta?

Caso estremamente semplice dove varia solo l'intercetta – le rette sono parallele

La regressione lineare tra una *variabile indipendente* X (predittore) ed una *variabile dipendente* Y (criterio) calcola la migliore approssimazione lineare che identifica il legame tra le due variabili utilizzando la struttura di dati bivariata (x_i, y_i) osservata su un campione di n osservazioni.

$$x = (x_1, x_2, \dots, x_n)$$

$$y = (y_1, y_2, \dots, y_n)$$

$$d = ((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$$

$$Y = \beta_0 + \beta X$$

$$y_i = \beta_0 + \beta x_i + \varepsilon_i \quad (i = 1, \dots, n)$$

y_i : Valore di y atteso per l' i -esima osservazione

β_0 : Valore dell'intercetta, ovvero *valore atteso in y quando $x_i = 0$*

β_1 : Coefficiente angolare ovvero la pendenza della retta. Definisce *il cambiamento medio atteso in y per ogni cambiamento unitario in x*

ε_i : Componente d'errore ($\mathcal{N}(0, \sigma^2)$)

3.6.2 Principio dei minimi quadrati

$$SQE = \sum_{i=1}^n (y_i - y_i^*)^2 = \sum_{i=1}^n (y_i - (\beta_0 + \beta x_i))^2 = \sum_{i=1}^n e_i^2$$

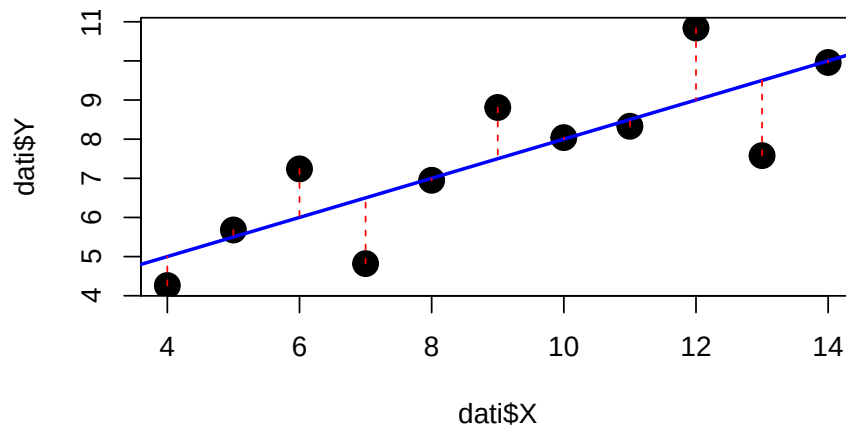
dove $y_i^* = \beta_0 + \beta x_i$, e $e_i = y_i - y_i^*$.

Per il principio dei minimi quadrati si può dimostrare che lo stimatore ottimale per β è

$$\hat{\beta} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

mentre per β_0 è

$$\hat{\beta}_0 = \bar{y} - \hat{\beta} \bar{x}$$



3.6.3 Proprietà della retta di regressione

3.6.3.1 Coefficiente angolare e correlazione

$$\hat{\beta} = \frac{\text{Cov}(X, Y)}{\text{sd}(x)^2}$$

$$r = \frac{\text{Cov}(X, Y)}{\text{sd}(x)\text{sd}(y)}$$

Moltiplicando e dividendo $\hat{\beta}$ per $\text{sd}(y)$:

$$\hat{\beta} = r \frac{\text{sd}(y)}{\text{sd}(x)}$$

1. Il **segno** del coefficiente angolare è lo stesso di r : $r > 0 \Rightarrow$ retta crescente, $r < 0 \Rightarrow$ retta decrescente, $r = 0 \Rightarrow$ retta orizzontale
2. La retta passa sempre per il **punto delle medie** $(\bar{x}, \bar{y})$ (basta sostituire $x = \bar{x}$ in $\hat{y} = \hat{\beta}_0 + \hat{\beta}x$)

3.6.3.2 Il bambino di prima

```
b <- cov(bimbo$eta, bimbo$peso) / var(bimbo$eta)
b0 <- mean(bimbo$peso) - b * mean(bimbo$eta)
c(b0, b)
```

```
[1] 5.1477273 0.3901515
```

```
cor(bimbo$eta, bimbo$peso) * sd(bimbo$peso) / sd(bimbo$eta)
```

```
[1] 0.3901515
```

Retta di regressione: $\hat{y} = 5.15 + 0.39x$

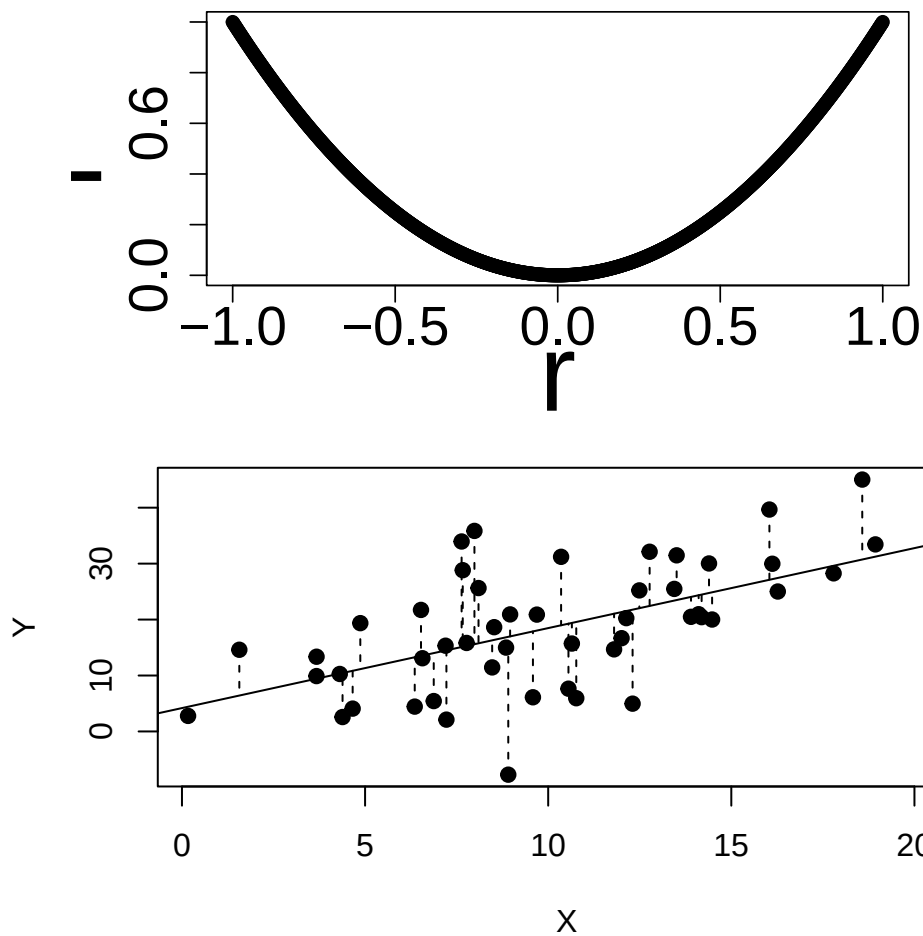
Quale sarebbe il peso previsto a 48 mesi? $\hat{y}_{48} = 5.15 + 0.39 \cdot 48 = 23.88$ kg

3.6.4 Indice di adattamento R^2

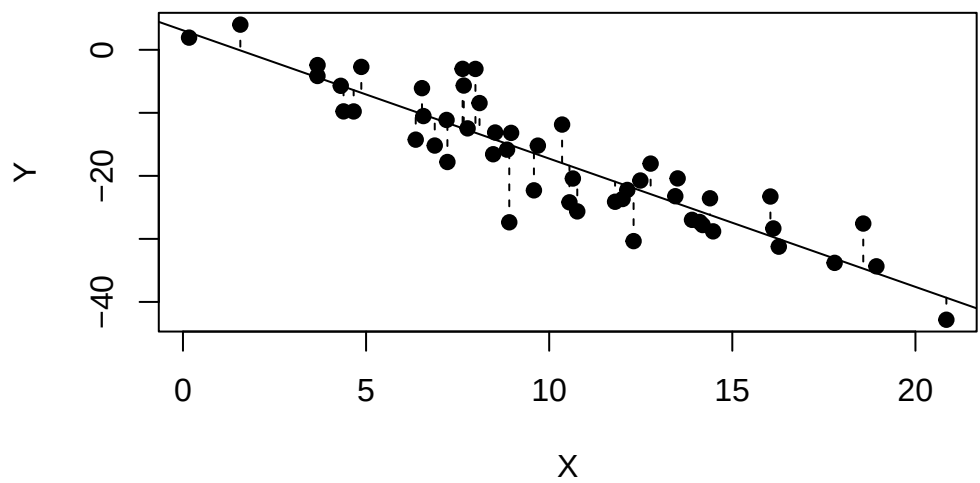
$$R^2 = 1 - \frac{SQE/n - 1}{Var(Y)} = 1 - \frac{\sum_{i=1}^n e_i^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

$\varepsilon = \frac{SQE}{n-1}$: Stimatore campionario della varianza della componente di errore

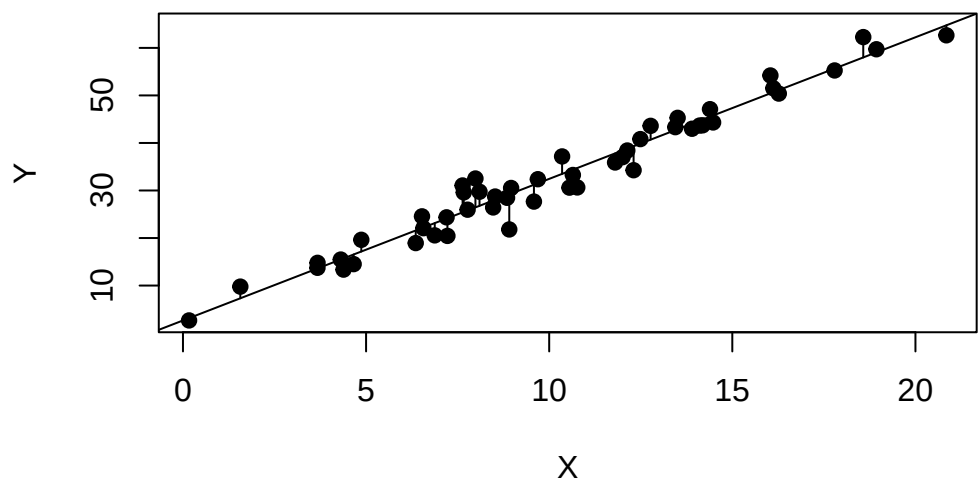
$0 \leq R^2 \leq 1$: Più R^2 è vicino ad 1, migliore è il modello. Di fatto, $R^2 = r^2$



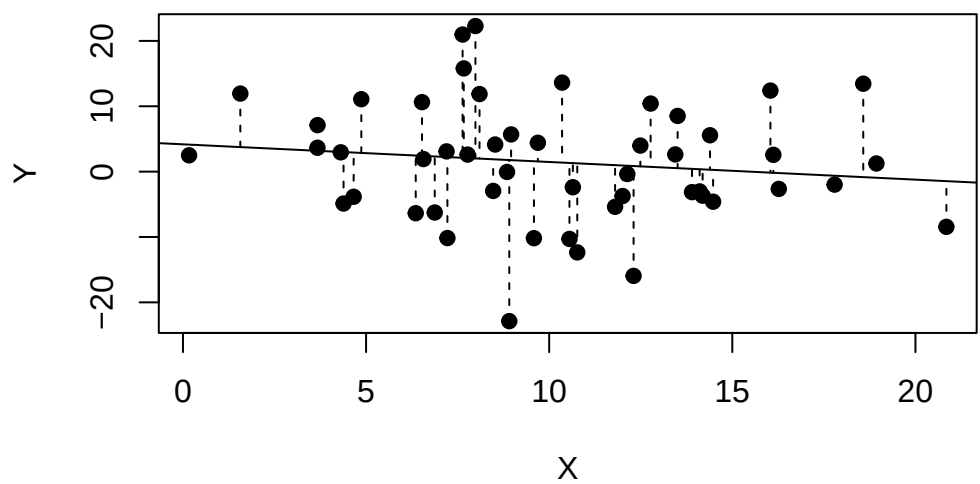
$R^2 = 0.34$, $r = 0.59$



$$R^2 = 0.81, r = -0.9$$



$$R^2 = 0.96, r = 0.98$$



$$R^2 = 0, r = -0.14$$

3.6.5 lm()

`lm(Y ~ X)`: Restituisce la stima dei parametri del modello di regressione lineare dove si predicono i valori di Y a seconda dei valori di X

`~`: (Per i PC con tastierino numerico): ALT + 126, Per i MAC: ALT + 5 (fonte non verificata)

```
X <- c(9,8,12,9,10,14,5,4,13,7,5)
Y <- c(9,6,8,9,7,10,8,4,11,5,4)
lm(Y~X)
```

Call:

```
lm(formula = Y ~ X)
```

Coefficients:

(Intercept)	X
2.4554	0.5624

3.6.6 Assunti del modello

3.6.6.1 Assunzioni

- **Indipendenza dei predittori dall'errore**

I predittori sono misurati senza errore, ovvero i predittori e gli errori di misura non sono correlati nella popolazione (per questo è importante il campionamento casuale)

- **Indipendenza delle osservazioni**

$\forall i \neq j, \varepsilon_i \perp \varepsilon_j$ (le osservazioni sono state campionate indipendentemente l'una dall'altra)

- **Linearità**

Il valore atteso dell'errore per un dato valore di x è 0

3.6.6.2 Assunzioni

- **Normalità**

$\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$: Anche y_i è distribuita normalmente con media $\beta_0 + \beta_1 x_i$

- **Varianza costante** (omoschedasticità)

La varianza degli errori è costante per qualunque valore di X

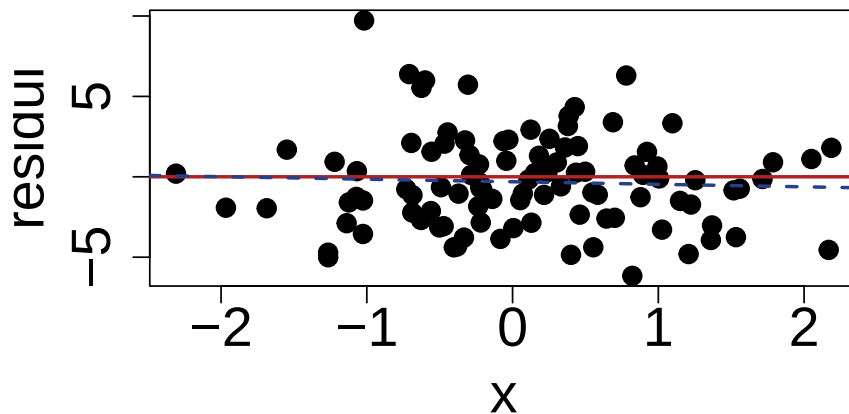
Gli errori ε_i della popolazione non si osservano $\rightarrow$ si guardano i loro “rappresentanti” nel campione: i **residui** $e_i = y_i - \hat{y}_i$

Indipendenza dei predittori dall'errore

I predittori sono misurati senza errore, ovvero i predittori e gli errori di misura non sono correlati nella popolazione (per questo è importante il campionamento casuale)

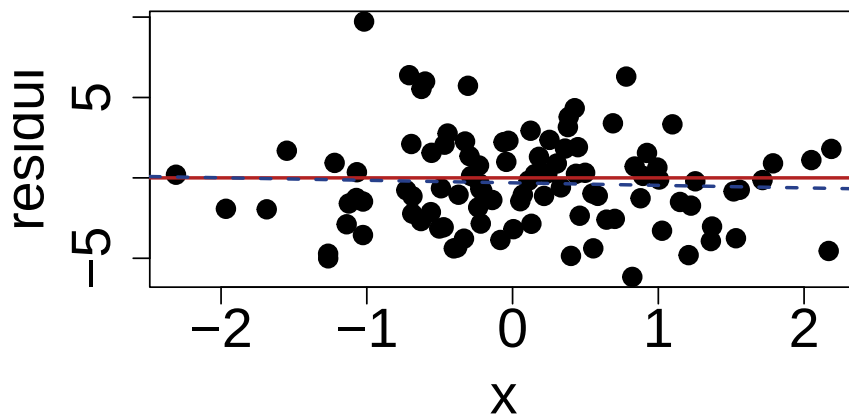
Analisi dei residui

I residui e il predittore **non devono essere correlati** tra di loro

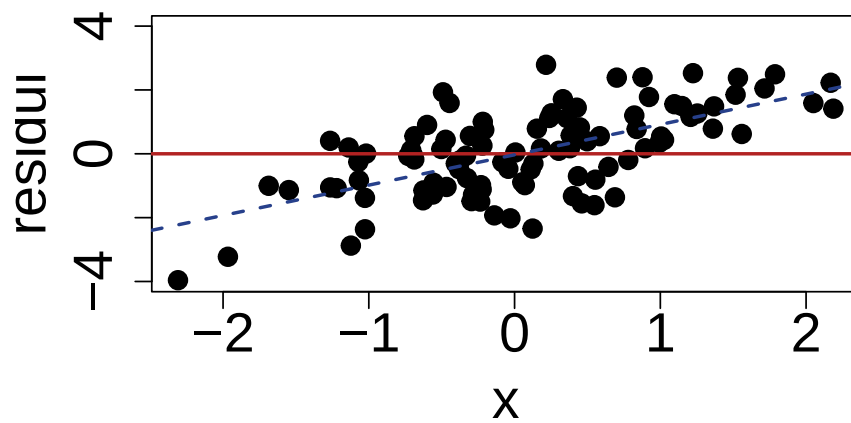
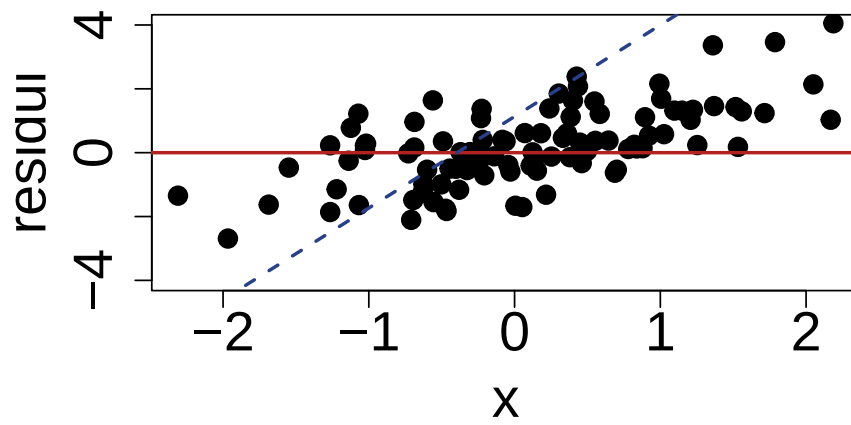


La linea intera rossa rappresenta quello che dovremmo osservare, ossia la totale assenza di relazione tra la x e i residui del modello

Le linea tratteggiata blu indica la relazione effettiva tra i residui e il predittore x . Questa linea deve essere il più vicina possibile alla linea rossa



$$r = -0.05$$



$$r = 0.64$$

```
head(studio)
```

```
      ore voto
1 18.3 26.4
2 18.7 25.5
3  5.7 19.9
4 16.6 23.5
5 12.8 19.9
6 10.4 22.1
```

```
modello <- lm(voto ~ ore, data = studio)
modello
```

Call:

```
lm(formula = voto ~ ore, data = studio)
```

Coefficients:

```
(Intercept)      ore
  14.8678      0.6243
```

3.6.6.3 I valori attesi e i residui

```
studio$attesi <- fitted(modello)      # y cappello
studio$residui <- residuals(modello)  # y - y cappello
head(studio)
```

```
   ore voto   attesi   residui
1 18.3 26.4 26.29279  0.1072084
2 18.7 25.5 26.54252 -1.0425177
3  5.7 19.9 18.42642  1.4735784
4 16.6 23.5 25.23146 -1.7314560
5 12.8 19.9 22.85906 -2.9590587
6 10.4 22.1 21.36070  0.7392975
```

```
# i residui sono semplicemente osservati - attesi
head(studio$voto - studio$attesi)
```

```
[1]  0.1072084 -1.0425177  1.4735784 -1.7314560 -2.9590587  0.7392975
```

3.6.6.4 Normalità degli errori

La media dei residui è (sempre) 0:

```
mean(studio$residui)
```

```
[1] -2.791617e-17
```

I residui dovrebbero distribuirsi in modo simmetrico attorno allo 0: la mediana deve essere il più vicina possibile a 0, il minimo e il massimo (così come il primo e il terzo quartile) dovrebbero essere simili in valore assoluto

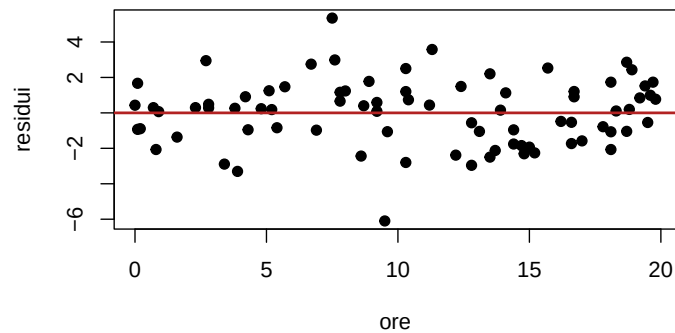
```
summary(studio$residui)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-6.0988 -1.0629   0.2153   0.0000  1.2028   5.3498
```

3.6.6.5 Residui vs predittore

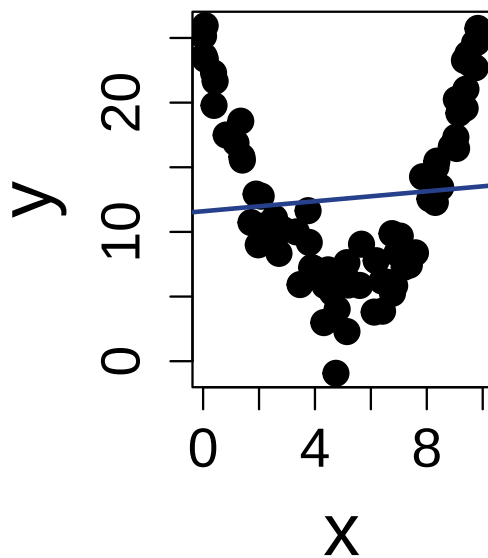
Se la relazione è lineare, i residui si distribuiscono **casualmente** attorno allo 0, senza alcun andamento

```
plot(studio$ore, studio$residui, pch = 19,
     xlab = "ore", ylab = "residui")
abline(h = 0, col = "firebrick", lwd = 2)
```

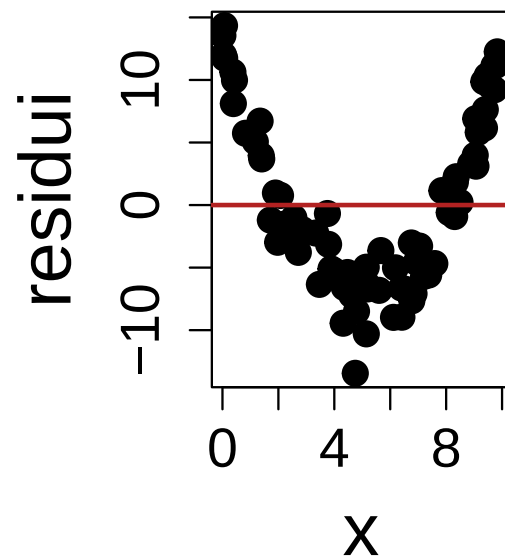


3.6.6.6 Quando la relazione non è lineare

Dati e retta



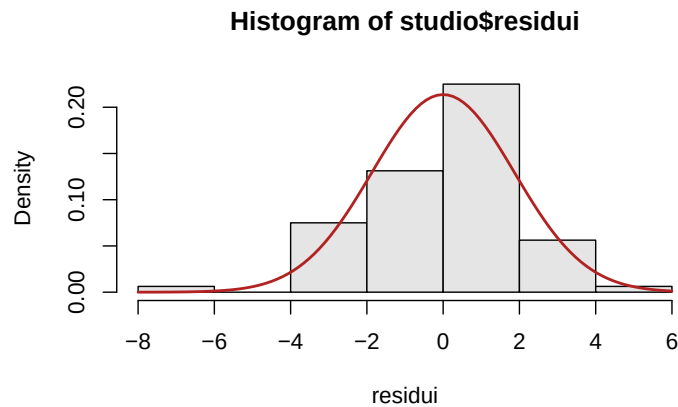
Residui



I residui hanno una chiara forma a “U”: il modello lineare sovrastima al centro e sottostima agli estremi → l’assunto di linearità è violato

3.6.6.7 Istogramma dei residui

```
hist(studio$residui, freq = FALSE, xlab = "residui", col = "gray90")
curve(dnorm(x, mean = 0, sd = sd(studio$residui)),
      add = TRUE, col = "firebrick", lwd = 2)
```

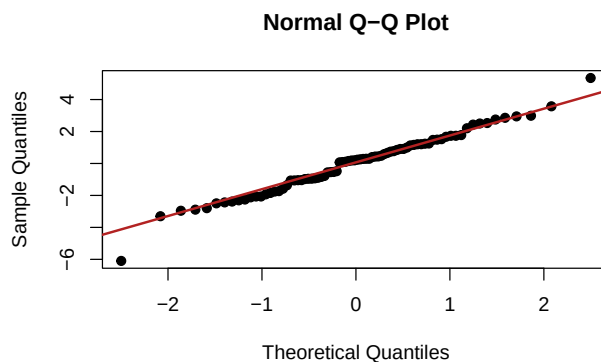


La linea rossa è la distribuzione normale con media 0 e sd pari a quella dei residui

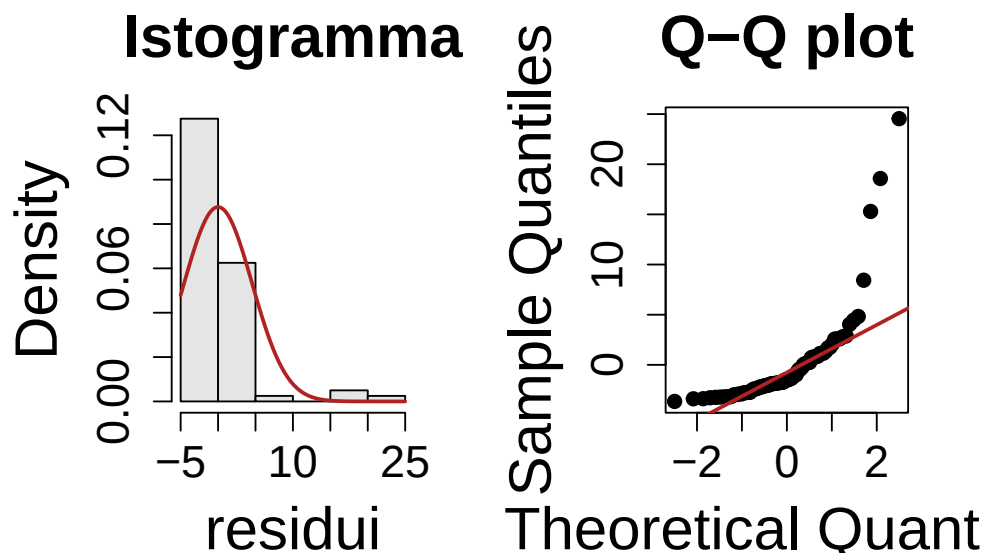
3.6.6.8 Q-Q plot dei residui

Confronta i quantili dei residui con quelli di una distribuzione normale: se i residui sono normali, i punti stanno **sulla retta**

```
qqnorm(studio$residui, pch = 19)
qqline(studio$residui, col = "firebrick", lwd = 2)
```



3.6.6.9 Quando i residui non sono normali

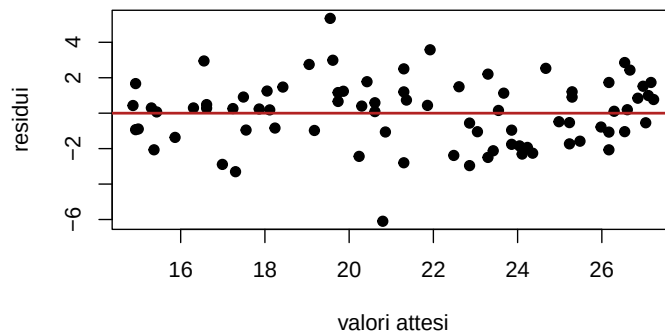


Distribuzione asimmetrica: coda lunga a destra, i punti si allontanano dalla retta agli estremi

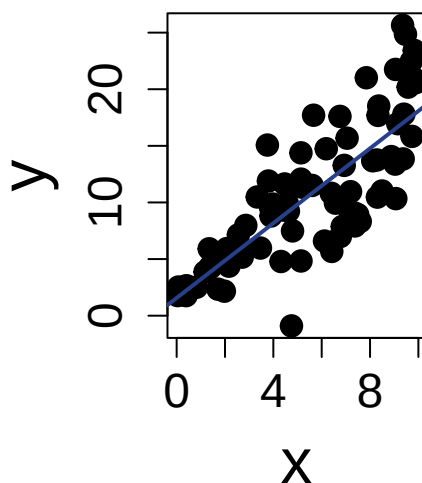
3.6.6.10 Omoschedasticità

La variabilità dei residui deve essere **costante** per tutti i valori attesi: la “nuvola” deve avere la stessa ampiezza da sinistra a destra

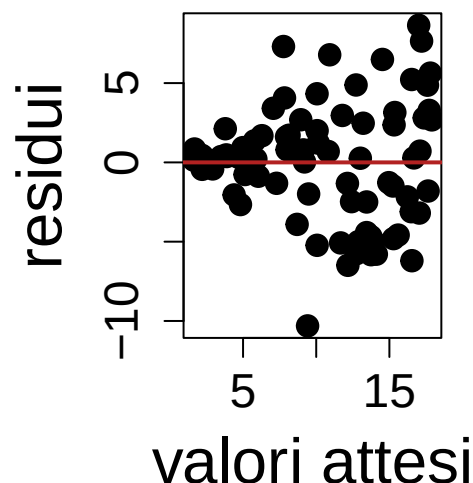
```
plot(studio$attesi, studio$residui, pch = 19,  
      xlab = "valori attesi", ylab = "residui")  
abline(h = 0, col = "firebrick", lwd = 2)
```



Dati e retta



Residui



Forma “a imbuto”: i residui sono sempre più dispersi al crescere dei valori attesi → eteroschedasticità

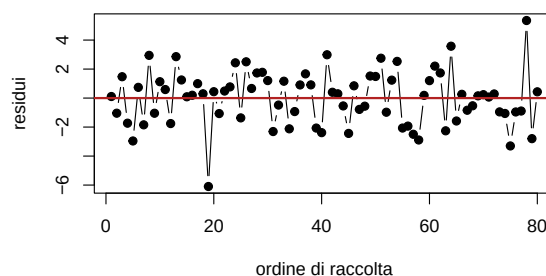
3.6.6.11 Indipendenza delle osservazioni

Dipende soprattutto da **come sono stati raccolti i dati**: ogni osservazione deve essere indipendente dalle altre

Esempi di violazione: misure ripetute sulla stessa persona, studenti della stessa classe, dati raccolti nel tempo

Se i dati hanno un ordine (es. ordine di raccolta), si possono guardare i residui in quell'ordine: non deve esserci nessun andamento

```
plot(studio$residui, type = "b", pch = 19,
     xlab = "ordine di raccolta", ylab = "residui")
abline(h = 0, col = "firebrick", lwd = 2)
```



3.6.6.12 Cosa guardare nei residui

Assunto	Grafico	Cosa si deve vedere
Linearità	<code>plot(x, residui)</code>	Nuvola casuale attorno a 0
Normalità	<code>hist(residui)</code> , <code>qqnorm(residui)</code>	Forma a campana, punti sulla retta
Omoschedasticità	<code>plot(attesi, residui)</code>	Ampiezza costante (niente imbuti)
Indipendenza	<code>plot(residui, type = "b")</code>	Nessun andamento nell'ordine

Tutto parte da due funzioni:

```
attesi <- fitted(modello)
residui <- residuals(modello)
```

3.7 Materiale supplementare

3.7.1 Outliers e casi influenti

3.7.1.1 Definizioni

Outlier: Il valore di un'osservazione nella variabile dipendente è anomalo dato il valore del predittore (ha un residuo molto grande)

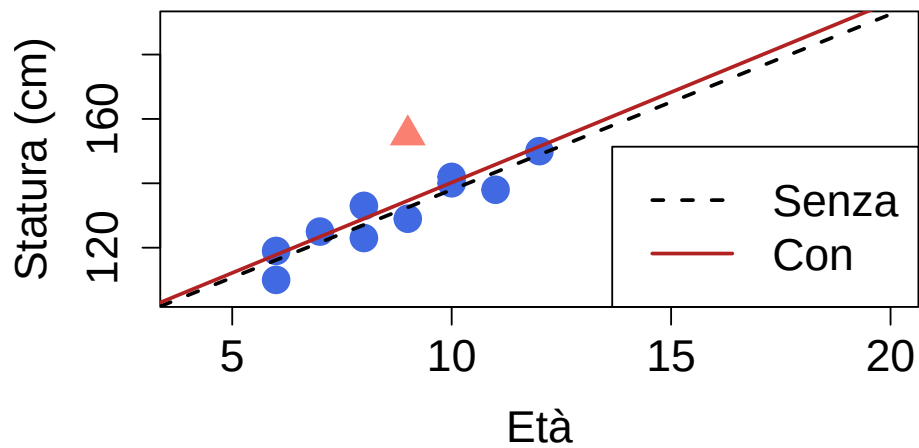
Leverage (leva): Quanto il valore di un'osservazione nel predittore è lontano dalla media del predittore → quanto *potenzialmente* può spostare la retta

Influential case: Una singola osservazione che riesce a spostare l'intera retta di regressione

Un'osservazione può essere:

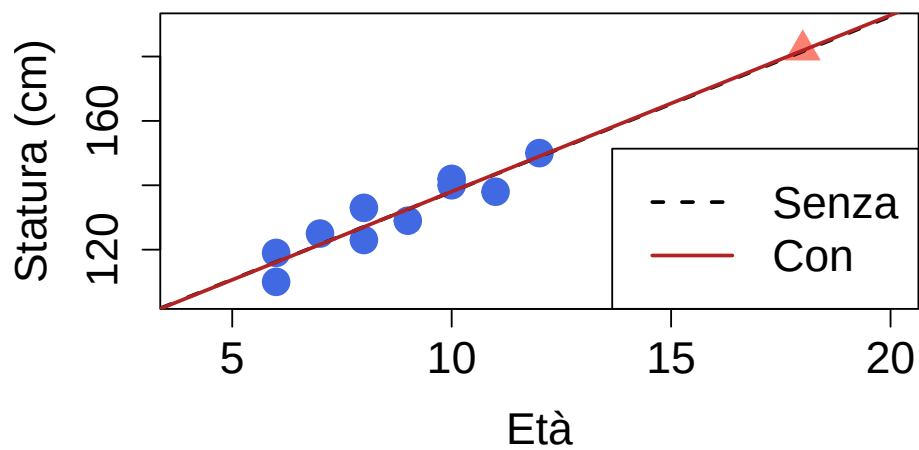
- Un outlier (ma non un influential case)
- Un punto ad alta leva in linea con gli altri (ma non un influential case)
- Un outlier influente (anomalo e ad alta leva)

3.7.1.2 Un outlier living the life



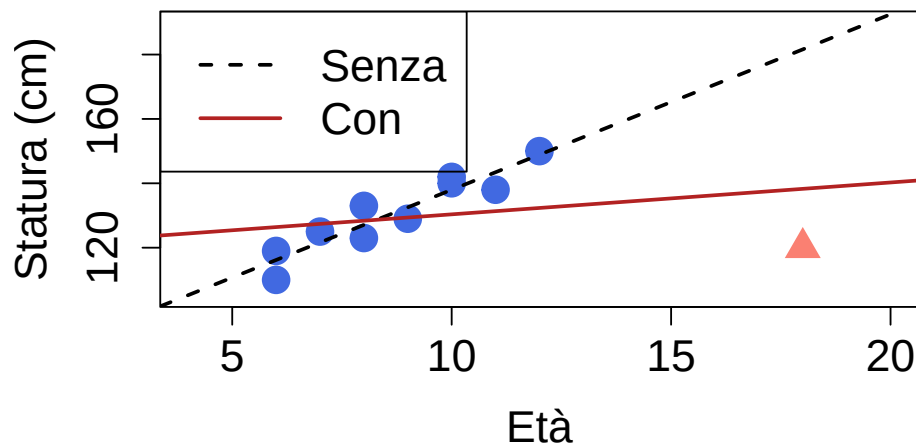
Il triangolo rosa è un outlier: è anomalo rispetto alla nuvola di punti, ma essendo vicino alla media dell'età sposta la retta di poco (soprattutto in alto, l'intercetta)

3.7.1.3 Un punto ad alta leva living the life



Il triangolo rosa è lontano dagli altri sull'asse delle x (alta leva), ma è in linea con l'andamento dei dati: la retta non cambia

3.7.1.4 Un outlier influential case living the life



Il triangolo rosa è anomalo e lontano dagli altri sull'asse delle x : ne sposta drasticamente la pendenza

3.7.2 Leverage e distanza di Cook

3.7.2.1 Tirare fuori leverage e distanza di Cook

Come per i residui, anche gli indici di influenza si estraggono dal modello e si guardano

`hatvalues()`: leverage di ogni osservazione

`cooks.distance()`: distanza di Cook $\rightarrow$ quanto cambiano *tutti* i valori attesi se l'osservazione viene tolta (combina residuo e leverage)

```
influyente <- rbind(dat, c(soggetto = 11, eta = 18, statura = 120))
mod_inf <- lm(statura ~ eta, data = influyente)
influyente$leva <- hatvalues(mod_inf)
influyente$cook <- cooks.distance(mod_inf)
```

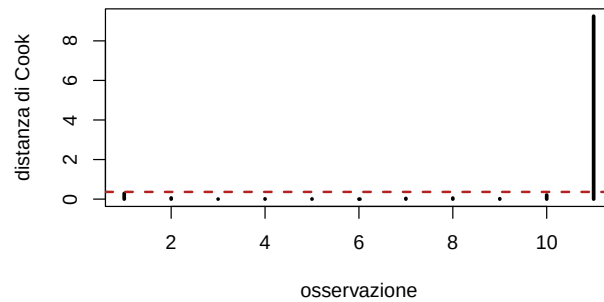
```
round(influyente, 2)
```

	soggetto	eta	statura	leva	cook
1	1	6	110	0.20	0.29
2	2	6	119	0.20	0.06
3	3	7	125	0.15	0.00
4	4	8	123	0.11	0.01
5	5	8	133	0.11	0.01
6	6	9	129	0.09	0.00
7	7	10	140	0.09	0.04
8	8	10	142	0.09	0.05
9	9	11	138	0.11	0.02
10	10	12	150	0.14	0.21
11	11	18	120	0.70	9.25

3.7.2.2 Il grafico della distanza di Cook

Una soglia spesso usata: $D_i > 4/n$

```
plot(influente$cook, type = "h", lwd = 3,  
      xlab = "osservazione", ylab = "distanza di Cook")  
abline(h = 4 / nrow(influente), col = "firebrick", lty = 2, lwd = 2)
```



```
which(influente$cook > 4 / nrow(influente))
```

```
[1] 11
```

4 Funzioni riassuntive dei moduli

4.1 Operatori

4.1.1 Aritmetici

Simbolo	Significato	Note
+	Addizione	
-	Sottrazione	
*	Moltiplicazione	
/	Divisione	
^	Elevamento a potenza	

4.1.2 Di confronto

Simbolo	Significato	Note
<	Minore	
>	Maggiore	
<=	Minore uguale	
>=	Maggiore uguale	
==	Uguale	
!=	Diverso	

4.1.3 Logici

Simbolo	Significato	Note
&	Congiunzione	Intersezione tra insiemi $\wedge$ ($\cap$)
	Disgiunzione	Unione tra insiemi $\vee$ ($\cup$)
!	Negazione	$\neg$
&!	Disgiunzione esclusiva	
%in%	Inclusione	

4.2 Valori di verità

Valore	Significato	Note
TRUE	Vero	Es. $X > 2$ confronta ogni valore di X con 2
FALSE	Falso	

4.3 Operatori e funzioni utili

Funzione	Significato	Note
<code><-</code> <code>c(...)</code>	Operatore di assegnazione Concatenazione di valori	Uguale a <code>assign()</code> oppure <code>=</code> <code>c(1:n)</code> restituisce valori da 1 a n con passo 1; <code>c(1,n)</code> restituisce solo 1 e n
<code>vettore[i]</code>	Operatore di selezione per i vettori, dove i è l'indice della posizione a cui si vuole accedere	Es. <code>A<-c(1,8,5,0)</code> ; <code>A[c(1,4)]</code> restituisce valori in posizione 1 e 4
<code>dati[i, j]</code>	Operatore di selezione per strutture di dati (matrici, dataframe), dove i è l'indice della riga, j della colonna	Es. <code>data[4, 12]</code> restituisce il valore della cella che si trova all'incrocio tra la quarta riga e la dodicesima colonna
<code>data\$variabile</code>	Visualizza la variabile richiamata	
<code>read.csv2("")</code>	Legge un file di dati esterno	<code>sep = ";"</code> , <code>dec = ","</code> , <code>header = TRUE</code>

4.4 Convertitori

Funzione	Significato	Note
<code>as.numeric(A)</code>	Converte in variabile numerica	
<code>as.factor(A)</code>	Converte in variabile categoriale/nominale	Richiede variabile testuale
<code>factor(A, levels=c("a1", ..., "an"), ordered=TRUE)</code>	Converte in variabile categoriale ordinale	Va inserito ordine dei livelli teorici
<code>as.matrix(data.frame(A))</code>	Converte un data.frame in oggetto numerico	

4.5 Strumenti

Funzione	Significato	Note
<code>remove(...)</code>	Rimuove valori assegnati	
<code>rm(...)</code>	Rimuove oggetti	
<code>rm(list=ls())</code>	Rimuove tutto ciò che è in memoria	
<code>ls()</code>	Visualizza oggetti in memoria	
<code>example(comando)</code>	Fornisce esempio d'uso	
<code>?comando</code>	Mostra guida	

4.6 Funzioni matematiche

Funzione	Significato	Note
<code>log(...)</code>	Logaritmo naturale	
<code>exp(...)</code>	Esponenziale	
<code>abs(...)</code>	Valore assoluto	
<code>sqrt(...)</code>	Radice quadrata	
<code>round(...)</code>	Arrotondamento	
<code>sign(...)</code>	Funzione segno	
<code>pi</code>	Costante pi greco	
<code>max(...)</code>	Valore massimo	
<code>min(...)</code>	Valore minimo	
<code>sum(...)</code>	Somma	
<code>mean(...)</code>	Media	
<code>mean(..., trim=n)</code>	Media troncata	Elimina outlier con $0 \leq n \leq 0.5$
<code>sd(...)</code>	Deviazione standard	
<code>var(...)</code>	Varianza	
<code>median(...)</code>	Mediana (2° quartile)	
<code>quantile(X, probs)</code>	Calcola quantile	
<code>IQR(...)</code>	Differenza interquartilica, $Q_3 - Q_1$	
<code>range(...)</code>	Intervallo (min e max)	Identico a <code>c(min(X), max(X))</code>
<code>rep(n, k)</code>	Replica n per k volte	
<code>dim(...)</code>	Dimensioni oggetto tabellare	<code>dim(...)[1]</code> righe, <code>dim(...)[2]</code> colonne
<code>cumsum(...)</code>	Somma cumulata	
<code>length(...)</code>	Numero elementi di un vettore	
<code>which(...)</code>	Posizione di valori che soddisfano condizione	Es. <code>which(x<3)</code>
<code>levels(...)</code>	Etichette assegnate a fattori	
<code>numeric(n)</code>	Vettore vuoto lunghezza n	
<code>rownames(...)</code>	Nomi righe	
<code>colnames(...)</code>	Nomi colonne	
<code>summary(...)</code>	Riassume contenuto	Anche modelli <code>lm(...)</code>
<code>str(X)</code>	Controlla struttura oggetto	
<code>cov(X,Y)</code>	Covarianza campionaria	
<code>cor(X,Y)</code>	Correlazione lineare	$-1 \leq r \leq 1$

4.7 Matrici

Funzione	Significato	Note
<code>matrix(y, r, c)</code>	Matrice $r \times c$ con valori y	Riempimento per colonne
<code>matrix(y, r, c, byrow=TRUE)</code>	Matrice	Riempimento per righe
<code>t(M)</code>	Trasposta di M	
<code>*</code>	Prodotto tra matrici	

4.8 Liste e Data Frame

Funzione	Significato	Note
<code>list()</code>	Costruisce lista di oggetti	Prima creare oggetti, poi richiamarli nella lista
<code>data.frame(...)</code>	Costruisce tabella di variabili	Variabili devono avere stessa lunghezza
<code>apply(DATI, MARGIN, FUN)</code>	Applica funzione per righe (1)/colonne (2)	Es. <code>apply(DATI, 1, sum)</code>
<code>cbind(...)</code>	Unisce colonne	Variabili devono avere stessa lunghezza
<code>rbind(...)</code>	Unisce righe	
<code>head(df)</code>	Mostra prime 6 righe dataframe	
<code>tail(df)</code>	Mostra ultime 6 righe dataframe	

4.9 Distribuzioni di frequenza

Funzione	Significato	Note
<code>table(A)</code>	Tabella frequenze di variabile	
<code>table(A,B)</code>	Tabella di contingenza	
<code>margin.table(T,margin=T/sum(T))</code>	Frequenze marginali	1 righe, 2 colonne
<code>prop.table(T,margin=T)</code>	Frequenze relative	
<code>prop.table(T,margin=T)</code>	Probabilità condizionate	
<code>cumsum(table(X))</code>	Frequenze cumulate assolute	
<code>cumsum(table(X)/length(X))</code>	Frequenze cumulate relative	
<code>table(...)/length(...)</code>	Frequenze relative	

4.10 Funzioni grafiche

Funzione	Significato	Note
<code>hist(X,breaks)</code>	Istogramma	<code>breaks=</code> Calcolo ampiezza bin

Funzione	Significato	Note
<code>boxplot(X)</code>	Boxplot	anche <code>boxplot(X,Y,names=c("X","Y"))</code>
<code>runif(N,0,1)</code>	N valori casuali presi da distribuzione uniforme	tra 0 e 1
<code>density(X,bw)</code>	Stima densità	Usare <code>plot(density(...))</code>
<code>par(mfrow=c(n1,n2))</code>	Imposta finestra grafica multipla	<code>n1</code> =righe, <code>n2</code> =colonne
<code>skew(...)</code>	Misura asimmetria	
<code>kurt(...)</code>	Misura curtosi	
<code>plot(X,Y)</code>	Grafico di dispersione	

4.11 Associazione variabili categoriali

Funzione	Significato	Note
<code>chisq.test(T)\$statistic</code>	Calcola chi-quadrato osservato	
<code>chisq.test(T)\$expected</code>	Calcola frequenze teoriche	

4.12 Regressione lineare

Funzione	Significato	Note
<code>lm(Y~X)</code>	Modello regressione lineare	Y dipendente, X indipendente
<code>abline(retta,...)</code>	Traccia retta sul grafico	<code>abline(lm(Y~X))</code>
<code>cor(X,Y)^2</code>	Bontà di adattamento (R^2)	
<code>retta\$residuals</code>	Residui modello	Es. <code>plot(X,E); abline(0,0)</code>